Compilatori

UniVR - Dipartimento di Informatica

Fabio Irimie, Legion

2° Semestre 2025/2026

Indice

0.1 Storia dei compilatori . . . 3

1 Compilazione **3**
1.1 Modelli ibridi . . . 3
1.2 Fasi della compilazione . . . 4
1.2.1 Lexical analysis . . . 5
1.2.2 Parsing . . . 6
1.2.3 Intermediate code generator . . . 7
1.2.4 Optimization . . . 7
1.2.5 Code generation . . . 8
1.2.6 Riassunto delle fasi . . . 8
1.2.7 Front end e back end . . . 9

2 Generazione del codice intermedio **10**
2.1 Rappresentazioni intermedie . . . 10
2.1.1 Static checking . . . 10
2.1.2 Three-address code . . . 11
2.2 Traduzione diretta dalla sintassi . . . 11
2.2.1 Attributi . . . 11
2.2.2 Albero sintattico annotato . . . 12
2.2.3 Schemi di traduzione . . . 13

3 Analisi lessicale **14**
3.1 Preprocessione . . . 14
3.2 Analisi lessicale . . . 15
3.2.1 Attributi dei token . . . 16
3.3 Lookahead . . . 16
3.3.1 Coppia di buffer . . . 17
3.4 Riconoscimento delle keyword . . . 18
3.4.1 Symbol table . . . 19
3.5 Espressioni regolari . . . 19
3.5.1 Definizione di linguaggio . . . 19
3.5.2 Operazioni sulle stringhe . . . 19
3.5.3 Operazioni sui linguaggi . . . 20
3.5.4 Definizione di espressioni regolari . . . 20
3.5.5 Precedenza degli operatori . . . 21
3.5.6 Regole algebriche per le espressioni regolari . . . 22
3.5.7 Notazione compatta . . . 22
3.6 Diagrammi di transizione . . . 23
3.6.1 Grammatica modello . . . 23
3.6.2 Stringhe riservate . . . 25
3.7 Architettura di un diagram-based lexer . . . 26
3.7.1 Lex: Lexical Analyzer Generator . . . 28
3.7.2 Architettura di un lexical analyzer generator . . . 28
3.7.3 Simulare un DFA . . . 29
3.7.4 Convertire un NFA in un DFA . . . 29
3.7.5 Simulare un NFA . . . 32
3.7.6 Trasformare espressioni regolari in NFA . . . 32

3.7.7 Minimizzare un DFA 36

4 Analisi sintattica (Parsing) 37
4.1 Definizione della sintassi 38
4.1.1 Derivazioni 39
4.2 Ambiguità 40
4.3 Associatività degli operatori 40
4.4 Precedenza degli operatori 41
4.5 Sintassi degli statements 42
4.6 Errori 43
4.6.1 Gestione degli errori 43
4.7 Top-down parsing (Recursive descent) 44
4.7.1 Predictive parsing 46
4.7.2 Funzione first 48
4.7.3 Ricorsione sinistra immediata 48
4.8 Predictive parsing 49
4.8.1 Grammatiche LL(k) 49
4.8.2 Left factoring 49
4.8.3 Tabella di parsing 50
4.8.4 Grammatiche LL(1) 53
4.8.5 Predictive parser 54
4.9 Bottom-up parsing 56
4.9.1 Shift-reduce parsing 57
4.9.2 Scegliere le azioni di shift e reduce 58
4.9.3 Riconoscere gli handle 59
4.9.4 LR parsing 61
4.9.5 Collezione canonica di una grammatica LR(0) 62
4.9.6 DFA per la decisione di shift e reduce 65
4.9.7 Algoritmo di parsing LR 66
4.9.8 Simple LR parser (SLR) 67
4.9.9 Parser LR(1) 70
4.9.10 Parser LALR 73
4.9.11 Regole di precedenza e associatività 75

0.1 Storia dei compilatori

All'inizio i programmi venivano scritti traducendo formule matematiche in codice macchina. Nel 1948 Nathaniel Rochester sviluppò il primo **assembler**, un programma che traduceva istruzioni mnemoniche in codice macchina.

Esempio 0.1. $1 + 2 = 3$ in assembly:

```
MOV R0, 1
MOV R1, 2
ADD R0, R1, R2
MOV R0, 64
STO R2, R0
```

Nel 1953 John Backus sviluppò il primo linguaggio interpretato, chiamato **Speedcoding** che esevuiva dalle 10 alle 20 volte più lentamente dell'assembly scritto a mano, ma era più facile da scrivere e leggere. Successivamente nel 1954-1957 sviluppò il primo **compilatore** per il linguaggio **Fortran** (FORmula TRANslation) che aveva performance paragonabili all'assembly scritto a mano. Questo linguaggio ha avuto un enorme impatto sulla programmazione:

- **Open programming**: La possibilità di programmare viene estesa da un numero limitato di matematici e scienziati a un pubblico più ampio di persone.
- **Portability**: Programmi di alto livello potevano essere ricompilati per un nuovo computer con modifiche minime. Questo ha rimosso la necessità del programmatore di preoccuparsi su che architettura il programma sarebbe stato eseguito.

1 Compilazione

Un compilatore è un programma che legge un altro programma scritto in un linguaggio di programmazione e lo traduce in un altro linguaggio equivalente chiamato **target**. Il compilatore traduce l'intero programma in un altro (solitamente linguaggio macchina) prima dell'esecuzione.

1. Prende l'intero programma in input
2. Lo traduce in un altro linguaggio (solitamente linguaggio macchina)
3. Genera un file eseguibile che prende input e produce output

Il compilatore può anche applicare un processo di ottimizzazione **globale** sul programma, migliorando le prestazioni del codice generato.

1.1 Modelli ibridi

Esistono anche modelli ibridi di compilatori che combinano compilazione e interpretazione:

- **Compilazione in bytecode** che viene poi interpretato da una macchina virtuale. Ad esempio Java con JVM.

  - **Bytecode**: Codice intermedio
  - **Macchina virtuale**: Software che esegue il bytecode

- **Compilazione Just-In-Time (JIT)**: Il codice viene compilato **durante l'esecuzione**, consentendo ottimizzazioni dinamiche basate sull'uso effettivo del programma.

1.2 Fasi della compilazione

Le fasi della compilazione preservano la struttura che Fortran ha introdotto. Ci sono due parti principali nel processo di compilazione:

- **Analisi**: Determina le operazioni implicate dal programma sorgente che sono rappresentate in una struttura ad albero (rappresentazione intermedia). L'informazione sul programma sorgente è salvata nella **symbol table**, che contiene informazioni su variabili, funzioni, tipi, ecc e permette di comunicare tra le fasi di compilazione. Questa fase è composta da:
  - **Lexical analysis**: Suddivide il codice sorgente (una lista di caratteri) in token (una lista di parole chiave, identificatori, operatori, ecc.)
  - **Parsing**: Analizza la struttura sintattica del programma, costruendo un albero sintattico astratto (AST) che rappresenta la struttura gerarchica del programma.
  - **Semantic analysis**: Verifica la correttezza semantica del programma, controllando tipi, dichiarazioni, e altre regole del linguaggio.
- **Sintesi**: Prende la rappresentazione intermedia ad albero e traduce tutte le operazioni nel programma target. Questa fase è composta da:
  - **Optimization**: Migliora le prestazioni del codice generato, ad esempio eliminando codice morto o riducendo il numero di istruzioni.
  - **Code generation**: Traduce l'AST ottimizzato in codice macchina o in un altro linguaggio target **senza cambiare la semantica del programma originale**.

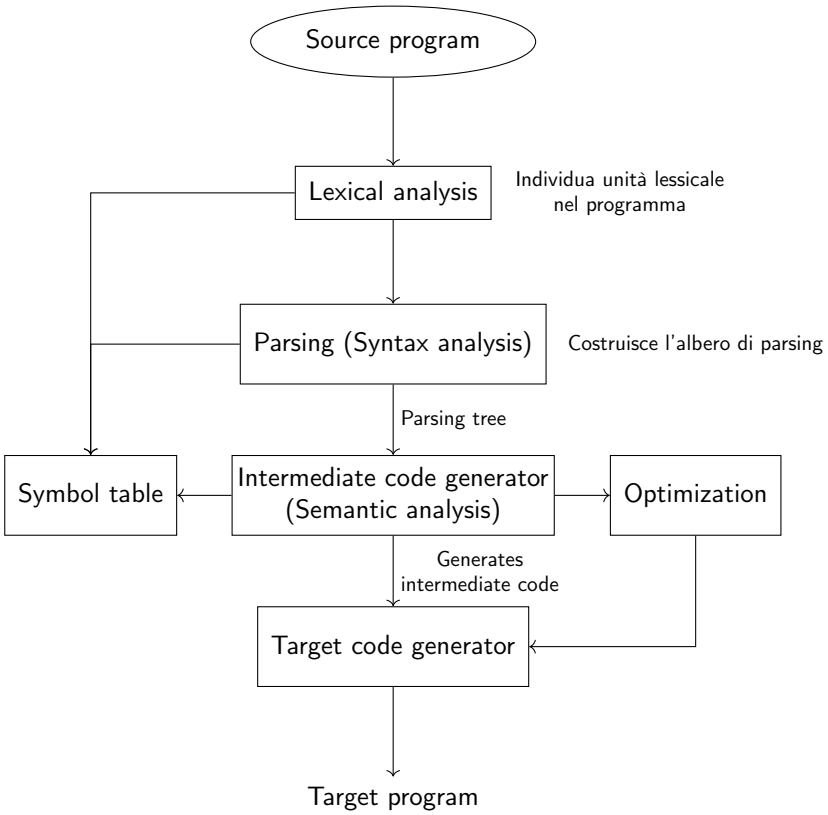


Figura 1: Struttura di un compilatore

Per generare codice eseguibile c'è bisogno di altri programmi in cui il compilatore è solo una parte del processo:

- **Preprocessor**: Prende in input il codice sorgente e lo trasforma, ad esempio espandendo macro o includendo file header.
- **Compilatore**: Traduce il codice sorgente preprocessato in codice oggetto (codice macchina non ancora collegato).
- **Assembler**: Traduce il codice oggetto in codice macchina eseguibile.
- **Linker**: Combina più file oggetto in un unico file eseguibile, risolvendo le dipendenze tra i vari moduli.

1.2.1 Lexical analysis

Il processo di lexical analysis, o tokenizzazione, è il primo passo nella fase di analisi e consiste nel riconoscere le "parole" del programma:

Esempio 1.1. Bisogna riconoscere:

```
This is a Compiler book.
```

Bisogna riconoscere anche che corrispondano alle regole del linguaggio, quindi la seguente stringa non è valida:

```
thi sis aCompil erb ook.
```

Il lexical analyzer legge lo stream di caratteri e li raggruppa in sequenze significative chiamate **lexemes** e per ogni lexeme produce in output un **token** della forma:

```
<token_name, token_value>
```

che viene messo nella tabella dei simboli.

Esempio 1.2. Consideriamo la seguente riga di codice:

```
position = initial + rate * 60
```

Il lexical analyzer raggruppa i caratteri nei seguenti lexemes e tokens:

Lexeme	Token
position	<id, 1>
=	<=>
initial	<id, 3>
+	<+>
rate	<id, 3>
*	<*>
60	<60>

Il `token_value` è un riferimento alla tabella dei simboli:

id	name
1	position
2	initial
3	rate

La sequenza di token ritornata dal lexical analyzer è:

```
<id, 1> <=> <id, 2> <+> <id, 3> <*> <60>
```

1.2.2 Parsing

Il parsing è una fase del lexical analysis che si occupa di analizzare la struttura sintattica del programma, costruendo un albero sintattico astratto (AST) che rappresenta la struttura gerarchica.

Esempio 1.3. La sequenza di token dell'esempio precedente viene rappresentata nel seguente albero sintattico:

```
<id, 1> <=> <id, 2> <+> <id, 3> <*> <60>
```

⇓

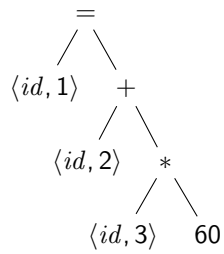


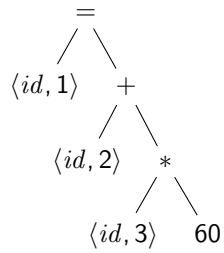
Figura 2: Syntax Tree

Questa fase controlla gli **errori semantici** (Type Checking) e ottiene le informazioni dei tipi per le fasi successive. Il type checking controlla i tipi degli operandi (potrebbe anche imporre delle conversioni di tipo) e controlla che le operazioni siano valide per i tipi degli operandi. Ad esempio controlla che non ci siano numeri reali per indicizzare un array.

1.2.3 Intermediate code generator

Il **codice intermedio** è generato come un programma per una **macchina astratta**. Questo codice dovrebbe essere facile da generare e tradurre nel programma target.

Esempio 1.4. L'intermediate code generator prende in input un albero sintattico e lo trasforma in codice intermedio:



⇓

```
temp1 = int2float(60)
temp2 = id3 * temp1
temp3 = id2 + temp2
id1 = temp3
```

1.2.4 Optimization

L'ottimizzazione **indipendentemente dalla macchina** cerca di migliorare il codice intermedio in modo da ottenere codice macchina migliore, ad esempio più veloce o che utilizza meno memoria o che consuma meno energia o rete, ecc... . Compilatori diversi possono applicare ottimizzazioni diverse e non tutte le ottimizzazioni sono

valide per tutti i programmi, ad esempio:
`x = y * 0` è uguale a 0 è:

- Valido per gli interi
- Non valido per i numeri in virgola mobile, perché dovrebbe essere uguale a `NaN`

Esempio 1.5. Un esempio di ottimizzazione è la seguente:

```
temp1 = int2float(60)
temp2 = id3 * temp1
temp3 = id2 + temp2
id1 = temp3
```

⇓

```
temp1 = id3 * 60.0
id1 = id2 + temp1
```

1.2.5 Code generation

Questa fase genera il codice target che consiste in codice assembly.

- Per ogni variabile e risultato intermedio, viene assegnato un **registro** o una posizione in memoria. L'**allocazione della memoria** viene fatta in questa fase, oppure nella fase di intermediate code generation.
- Le istruzioni sono tradotte in sequenze di istruzioni assembly.

Esempio 1.6. Il seguente codice verrà poi tradotto in assembly:

```
temp1 = id3 * 60.0
id1 = id2 + temp1
```

⇓

```
LDF R2, id3
MULF R2, R2, #60.0
LDF R1, id2
ADDF R1,R1, R2
STF id1, R1
```

1.2.6 Riassunto delle fasi

Esempio 1.7. Mettendo insieme tutte le fasi di compilazione si ottiene:

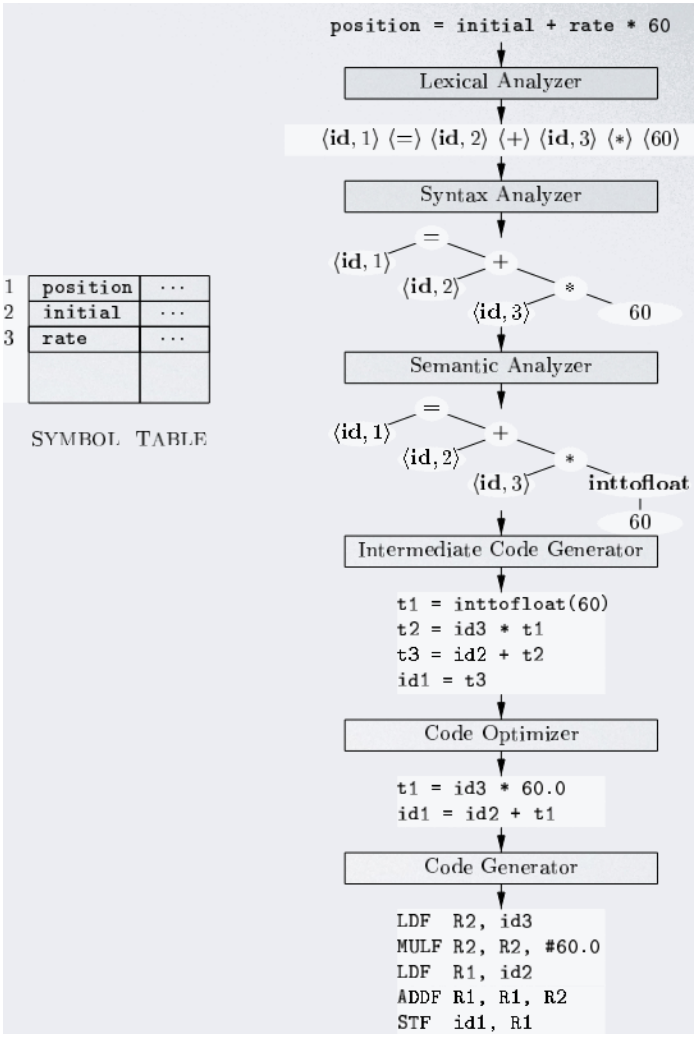


Figura 3: Riassunto delle fasi di compilazione

1.2.7 Front end e back end

- **Front-end**: Si occupa dell'analisi, cioè definisce la **sintassi** del linguaggio di programmazione (attraverso grammatiche context free).
- **Back-end**: Si occupa della sintesi.

Il front end è indipendente dalla macchina, mentre il back end è specifico per la macchina target. Questo permette di riutilizzare il front end per diversi target, ad esempio un compilatore che supporta sia x86 che ARM può avere lo stesso front end e due back end diversi.

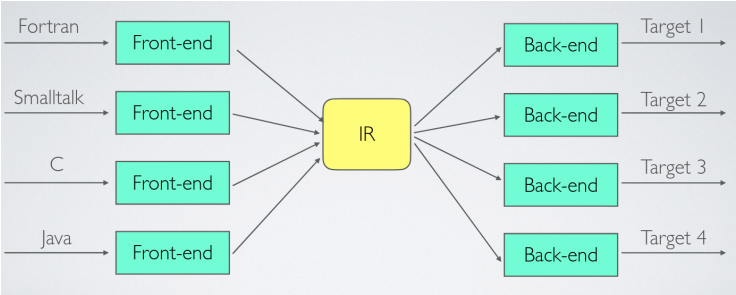


Figura 4: Esempi di front end e back end

2 Generazione del codice intermedio

2.1 Rappresentazioni intermedie

Le rappresentazioni intermedie più importanti sono:

- **Alberi**, inclusi gli alberi di parsing e gli alberi sintattici astratti (AST).
- **Rappresentazioni lineari**, in particolare il three-address code.

Oltre a creare una rappresentazione intermedia, il front end di un compilatore controlla che il programma sorgente segua le regole sintattiche e semantiche del linguaggio sorgente. Questo controllo è chiamato **static checking**.

Il codice intermedio è codice che rappresenta le operazioni del programma in modo più vicino al codice macchina, ma ancora indipendente dalla macchina target.

2.1.1 Static checking

Lo static checking applica controlli di consistenza e assicura che certi tipi di errori di programmazione, inclusi gli errori di tipizzazione, siano rilevati e segnalati durante la compilazione. Ci sono due tipi di static checking:

- **Syntactic checking**: Controlla che il programma sorgente segua le regole sintattiche del linguaggio, ad esempio che in un'assegnazione ci sia un identificatore a sinistra e un'espressione a destra:

```
# L-value   R-value
        x = 5; // corretto
        5 = x; // errore di sintassi

```

- **Type checking**: Controlla che le operazioni siano applicate a tipi di dati compatibili, ad esempio

```
int x = 5; // corretto
x = "hello"; // errore di tipo

```

2.1.2 Three-address code

Come codice intermedio consideriamo il **three-address code**, cioè codice simile all'assembly: una sequenza di istruzioni con al massimo tre operandi tali che:

- C'è al massimo un operatore, oltre all'assegnazione. In questo modo si rende esplicita la precedenza degli operatori.
- Bisogna generare nomi temporanei per memorizzare i risultati intermedi.

Esempio 2.1. Alcuni esempi di three-address code sono i seguenti:

- ```
  x = y op z
  ```
- ```
  x[y] = z
  x = y[z]
  ```
- ```
  ifFalse x goto L # se x e' falso salta a L
  ifTrue x goto L # se x e' vero salta a L
  goto L # salta a L
  ```

2.2 Traduzione diretta dalla sintassi

La traduzione diretta dalla sintassi è un metodo per generare codice intermedio direttamente dall'albero sintattico.

Utilizza una grammatica context free per specificare la struttura sintattica del linguaggio e associa un insieme di attributi ai terminali e non terminali della grammatica, associando ad ogni produzione un insieme di regole semantiche per calcolare i valori degli **attributi**. L'albero sintattico annotato viene poi attraversato e le regole semantiche applicate: dopo che l'attraversamento dell'albero è completato, i valori degli attributi sui non terminali contengono la forma tradotta dell'input.

2.2.1 Attributi

Un attributto è detto:

- **Synthesized** se il suo valore in un nodo dell'albero sintattico è determinato dai valori degli attributi nei figli del nodo (attraversamento depth-first).
- **Inherited** se il suo valore in un nodo dell'albero sintattico è determinato dal genitore (applicando le regole semantiche del genitore) e dai fratelli.

Esistono due tipi di notazioni:

- **Notazione infissa**: Gli operatori si trovano tra gli operandi, ad esempio:
  - $(9-5)+2$
  - $9-(5+2)$
- **Notazione postfissa**: Gli operatori si trovano dopo gli operandi, ad esempio:
  - $95-2+$
  ```

– $952 + -$

Questo tipo di notazione è più facile da tradurre in codice intermedio, perché non richiede di tenere conto della precedenza degli operatori.

Esempio 2.2. Grammatica per la traduzione da notazione infissa a postfissa:

- **Produzioni** (notazione infissa):

$$\begin{aligned} \mathrm{expr} &\rightarrow \mathrm{expr1} + \mathrm{term} \\ \mathrm{expr} &\rightarrow \mathrm{expr1} - \mathrm{term} \\ \mathrm{expr} &\rightarrow \mathrm{term} \\ \mathrm{term} &\rightarrow 0|1 \qquad\qquad |\ldots|9 \end{aligned}$$

- **Regole semantiche** (notazione postfissa):

$$\begin{aligned} \mathrm{expr.t} &\rightarrow \mathrm{expr1.t}||\mathrm{term.t}||" + " \\ \mathrm{expr.t} &\rightarrow \mathrm{expr1.t}||\mathrm{term.t}||" - " \\ \mathrm{expr.t} &\rightarrow \mathrm{term.t} \\ \mathrm{term.t} &\rightarrow "0" \\ \mathrm{term.t} &\rightarrow "1" \\ &\ldots \\ \mathrm{term.t} &\rightarrow "9" \end{aligned}$$

dove $||$ rappresenta la concatenazione di stringhe.

2.2.2 Albero sintattico annotato

L'albero sintattico annotato è un albero sintattico in cui ogni nodo è annotato con i valori degli attributi calcolati dalle regole semantiche.

Esempio 2.3. Un esempio di albero sintattico annotato è il seguente:

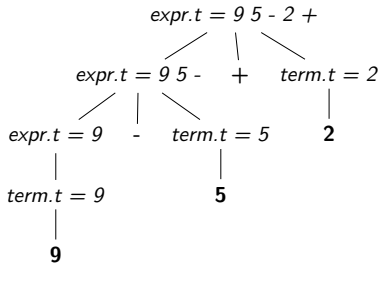


Figura 5: Albero sintattico annotato

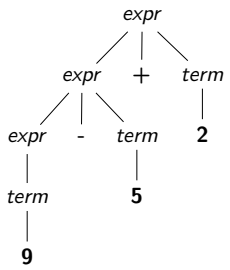


Figura 6: Albero sintattico

Un albero viene visitato tramite un **Depth-first traversal** in cui si visitano prima i figli di un nodo e poi si valutano le regole semantiche per quel nodo:

```
procedure visit(N) {
  for ( each child C of N , from left to right ) {
    visit(C);
  }
  evaluate the semantic rules at node N
}
```

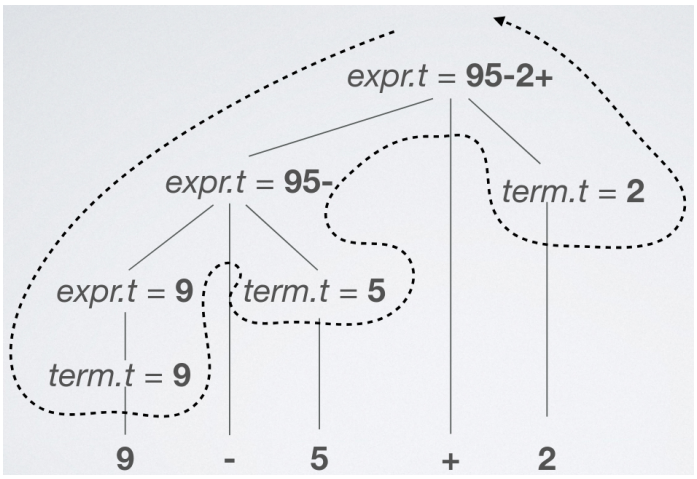


Figura 7: Depth-first traversal

Esempio 2.4.

2.2.3 Schemi di traduzione

Uno schema di traduzione è una grammatica context free in cui sono inserite delle **azioni semantiche**, cioè frammenti di codice che vengono eseguiti quando viene applicata una produzione.

Esempio 2.5. Un esempio è:

$$
\begin{array}{ll}
 & \overbrace{\qquad\qquad}^{\text{azione semantica}} \\
\text{expr} \to \text{expr1} + \text{term} & \{\text{print}(\text{"}+\text{"})\} \\
\text{expr} \to \text{expr1} - \text{term} & \{\text{print}(\text{"}-\text{"})\} \\
\text{expr} \to \text{term} & \\
\text{term} \to 0 & \{\text{print}(\text{"0"})\} \\
\text{term} \to 1 & \{\text{print}(\text{"1"})\} \\
\quad \ldots & \\
\text{term} \to 9 & \{\text{print}(\text{"9"})\}
\end{array}
$$

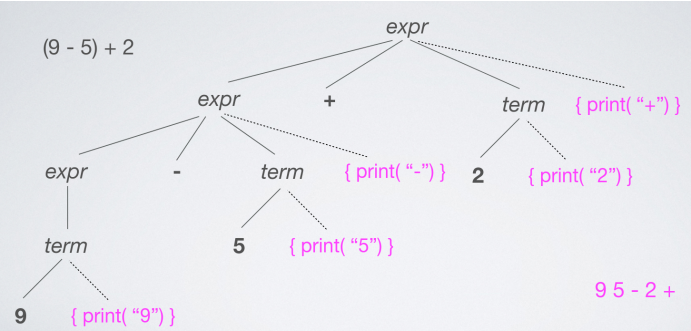


Figura 8: Esempio di schema di traduzione

3 Analisi lessicale

L'analizzatore lessicale, o lexer si occupa di suddividere il codice sorgente in token e di costruire la tabella dei simboli. Il lexer è solitamente implementato come un programma che legge il codice sorgente e produce in output una sequenza di token.



Figura 9: Esempio di analizzatore lessicale

L'analisi lessicale è una fase di compilazione separata per i seguenti motivi:

- Semplicità di design
- Permette di implementare ottimizzazioni mirate e quindi migliorare l'efficienza
- Portabilità del compilatore migliorata

3.1 Preprocessione

La preprocessione è una fase che avviene prima dell'analisi lessicale e si occupa di trasformare il codice sorgente in una forma più adatta per l'analisi lessicale, ad esempio rimuovendo spazi e commenti.

Esempio 3.1. Per rimuovere spazi e commenti si può utilizzare una funzione del tipo:

```
for (; ; peek = next input character) {
  if (peek is a blank or a tab) do nothing;
  else if (peek is a newline) line = line + 1;
  else break;
}
```

3.2 Analisi lessicale

L'analisi lessicale si occupa di riconoscere **unità lessicali** e trasformarle in **token**. Ad esempio il seguente codice sorgente:

```
1 if (i == j)
2   z = 0;
3 else
4   z = 1;
```

viene visto come una stringa di caratteri:

```
\tif (i == j) \n\t\tz = 0 ;\n \telse \n\t\tz = 1;
```

e poi viene suddiviso ad esempio nei seguenti tipi di token: **ws**, **keyword**, **id**, **num**, **op**, **(**, **)**, **;**, ecc...

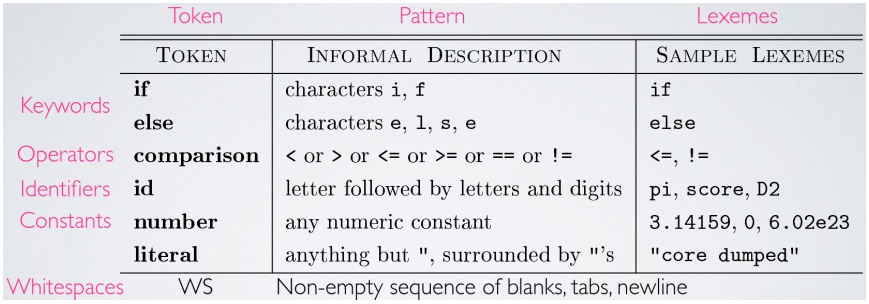
```
1 <ws, \t> <keyword, if> <ws,  > <(> <id, i> <ws,  > <op, ==>
2 <ws,  > <id, j> <)> <ws,  > <ws, \n\t\t> <id, z> <ws,  > <=>
3 <ws,  > <num, 0> <;> <ws, \n \t> <keyword, else> <ws, \n\t\t>
4 <id, z> <ws,  > <=> <num, 1> <;>
```

Definizione 3.1 (Token)**.** Un token è una coppia che consiste nel nome del token e un valore opzionale associato. Ad esempio, il token per la parola chiave `if` potrebbe essere rappresentato come $\langle$**if**, `if`$\rangle$, mentre il token per un numero potrebbe essere rappresentato come $\langle$**num**, 42$\rangle$.

Definizione 3.2 (Pattern)**.** Un pattern è una descrizione della forma che i lessemi di un token possono assumere. Ad esempio, il pattern per un numero potrebbe essere descritto come una sequenza di cifre, mentre il pattern per un identificatore potrebbe essere descritto come una lettera seguita da zero o più lettere o cifre.

Definizione 3.3 (Lessema)**.** Un lessema è una sequenza di caratteri del programma sorgente che corrisponde a un pattern di un token ed è identificato dal lexer come un'istanza di quel token. Ad esempio, nella stringa `if (i == j)`, i lessemi sono `if`, `(`, `i`, `==`, `j` e `)`.

Esempio 3.2. Alcuni esempi di tokens, patterns e lessemi sono i seguenti:



TOKEN	INFORMAL DESCRIPTION	SAMPLE LEXEMES
if	characters `i`, `f`	`if`
else	characters `e`, `l`, `s`, `e`	`else`
comparison	`<` or `>` or `<=` or `>=` or `==` or `!=`	`<=`, `!=`
id	letter followed by letters and digits	`pi`, `score`, `D2`
number	any numeric constant	`3.14159`, `0`, `6.02e23`
literal	anything but `"`, surrounded by `"`'s	`"core dumped"`
WS	Non-empty sequence of blanks, tabs, newline	

Figura 10: Esempi di tokens, patterns e lessemi

3.2.1 Attributi dei token

Più di un lessema può matchare lo stesso pattern, quindi il lexer associa ad ogni token un **attributo** (opzionale) che descrive il lessema specifico che è stato riconosciuto. Il nome del token influisce sulle decisioni di parsing, mentre il valore dell'attributo influenza la traduzione dei token dopo il parsing.

3.3 Lookahead

Il lookahead è una tecnica utilizzata per gestire i casi in cui un token può essere composto da più caratteri. Ad esempio i numeri possono essere composti da più cifre e le parole chiave possono essere composte da più lettere. In questi casi, il lexer deve leggere più caratteri per determinare se si tratta di un token valido o meno.

Esempio 3.3. Per gestire i numeri si può utilizzare una funzione del tipo:

```
1 if ( peek holds a digit ) {
2   v = 0;
3   do {
4     v = v * 10 + integer value of digit peek;
5     peek = next input character;
6   } while ( peek holds a digit );
7   return token ⟨num, v⟩;
8 }
```

Un altro esempio in cui è necessario il lookahead è quando le parole chiave del linguaggio non sono riservate, cioè quando una parola chiave può essere utilizzata anche come identificatore. In quel caso bisogna leggere più caratteri per determinare se si tratta di una parola chiave o di un identificatore.

L'**operatore di lookahead** è un operatore che viene utilizzato in un pattern del tipo r1/r2 quando il pattern r1 per un particolare token deve descrivere un contesto finale r2 per identificare correttamente il lessema effettivo. Ad esempio:

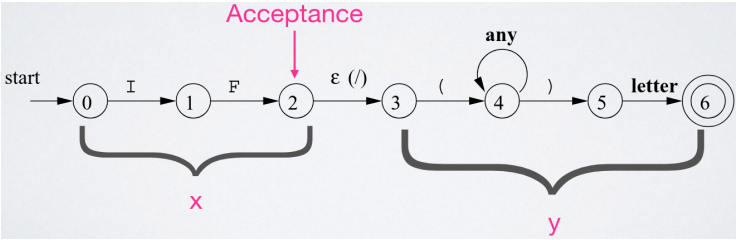


Figura 11: Esempio di operatore di lookahead

3.3.1 Coppia di buffer

Per gestire il lookahead, il lexer può utilizzare una coppia di buffer che vengono caricati alternativamente con i caratteri del programma sorgente. Si sfruttano due puntatori per analizzare i lessemi:

- `lexemeBegin`: punta al primo carattere del lessema corrente
- `forward`: avanza finchè non viene matchato un pattern, a quel punto viene ritratto

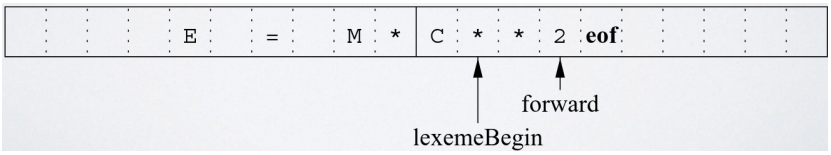


Figura 12: Esempio di coppia di buffer

Per riconoscere quando si è arrivati alla fine del buffer, oppure quando si è arrivati alla fine del file si possono utilizzare dei caratteri speciali, ad esempio il carattere `EOF` (End Of File).

Esempio 3.4. Un esempio di codice per riconoscere la fine del buffer o del file è il seguente:

```
switch ( *forward++ ) {
    case eof:
        if (forward is at end of first buffer ) {
            reload second buffer;
            forward = beginning of second buffer;
        }
        else if (forward is at end of second buffer ) {
            reload first buffer;
            forward = beginning of first buffer;
        }
        else /* eof within a buffer marks the end of input */
            terminate lexical analysis;
        break;
```

```
    Cases for the other characters
}
```

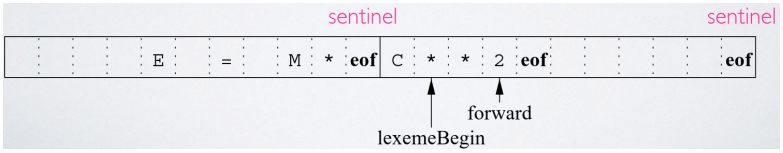


Figura 13: Esempio di riconoscimento della fine del buffer o del file

3.4 Riconoscimento delle keyword

Il riconoscimento delle keyword è una tecnica utilizzata per distinguere le parole chiave dalle identificatori. Ad esempio, la parola chiave `if` deve essere riconosciuta come una keyword e non come un identificatore. Per fare questo, il lexer può utilizzare una **symbol table** (tabella hash) che contiene tutte le parole chiave del linguaggio e confrontare ogni token con le parole chiave nella symbol table.

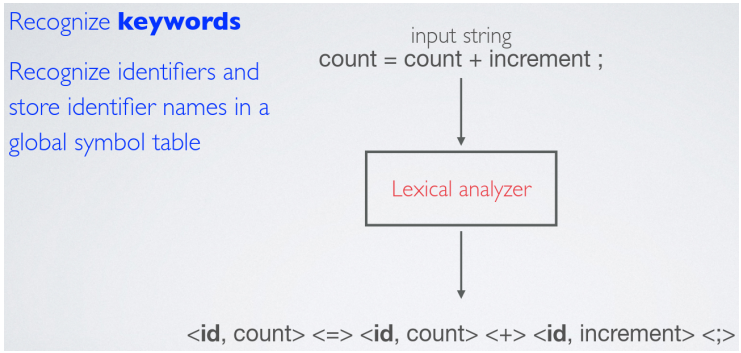


Figura 14: Esempio di riconoscimento delle keyword

Esempio 3.5. Per riconoscere le keyword si può utilizzare una funzione del tipo:

```
if ( peek holds a letter ) {
  collect letters or digits into a buffer b;
  s = string formed from the characters in b;
  w = token returned by words.get(s);
  if ( w is not null ) {
    return w;
  } else {
    Enter the key–value pair (s, ⟨id, s⟩) into words
    return token ⟨id, s⟩;
  }
```

```
11 }
```

Un altro compito del preprocessore è quello di correlare gli **errori generati dal compilatore** con il programma sorgente. Oppure se il programma sorgente utilizza le **macro**, la loro espansione è un compito del preprocessore.

3.4.1 Symbol table

La symbol table tipicamente deve supportare più dichiarazioni dello stesso identificatore all'interno di un programma. Lo scope di una dichiarazione è la porzione di un programma a cui si applica la dichiarazione e gli scope sono implementati creando una symbol table separata per ogni scope.

3.5 Espressioni regolari

Un'espressione regolare è una notazione per descrivere un insieme di stringhe che appartengono a un linguaggio regolare. Le espressioni regolari sono utilizzate per descrivere i pattern dei token che il lexer deve riconoscere.

3.5.1 Definizione di linguaggio

- **Alfabeto** Σ: è un insieme finito di simboli
- **Stringa** s: è una sequenza finita di simboli dell'alfabeto
  - $|s|$ rappresenta la lunghezza della stringa s
  - ε rappresenta la stringa vuota, cioè la stringa di lunghezza zero
- **Linguaggio** L: è un insieme di stringhe sull'alfabeto Σ

3.5.2 Operazioni sulle stringhe

Definizione utile 3.1. Alcuni termini importanti sono i seguenti:

- Il **prefisso** di una stringa s è qualsiasi stringa ottenuta rimuovendo zero o più simboli alla fine di s. Ad esempio, *ban*, *banana* e ε sono prefissi di *banana*.
- Il **suffisso** di una stringa s è qualsiasi stringa ottenuta rimuovendo zero o più simboli all'inizio di s. Ad esempio, *nana*, *banana* e ε sono suffissi di *banana*.
- La **sottostringa** di una stringa s è ottenuta rimuovendo qualsiasi prefisso o suffisso da s. Ad esempio, *nan* e ε sono sottostringhe di *banana*.
- I prefissi, suffissi e sottostringhe si dicono **propri** se sono diversi da s stesso o da ε.
- Una **sottosequenza** di una stringa s è una stringa ottenuta rimuovendo zero o più simboli da s anche in posizioni non consecutive. Ad esempio *baan* è una sottosequenza di *banana*.

Le operazioni sulle stringhe sono le seguenti:

- **Concatenazione**: La concatenazione di due stringhe x e y è denotata con xy
  - L'elemento nullo della concatenazione è ε:

$$s\varepsilon = \varepsilon s = s$$

- La ripetizione di una stringa s è denotata con s^n:

$$s^0 = \varepsilon$$
$$s^i = s^{i-1}s \quad \text{per } i > 0$$

3.5.3 Operazioni sui linguaggi

Le operazioni sui linguaggi sono le seguenti:

- **Unione**: $L \cup M = \{s \mid s \in L \vee s \in M\}$
- **Concatenazione**: $LM = \{xy \mid x \in L, y \in M\}$
- **Ripetizione**: $L^0 = \{\varepsilon\}; L^i = L^{i-1}L$ per $i > 0$
- **Chiusura di Kleene**: $L^* = \bigcup_{i \geqslant 0} L^i$
- **Chiusura positiva**: $L^+ = \bigcup_{i > 0} L^i$

3.5.4 Definizione di espressioni regolari

Le espressioni regolari sono definite ricorsivamente partendo da simboli base:

- ε è un'espressione regolare che denota il linguaggio $\{\varepsilon\}$
- Ogni simbolo a dell'alfabeto Σ è un'espressione regolare che denota il linguaggio $\{a\}$

Se r e s sono espressioni regolari che denotano i linguaggi $L(r)$ e $L(s)$ rispettivamente, allora:

- $r \mid s$ è un'espressione regolare che denota il linguaggio $L(r) \cup L(s)$
- rs è un'espressione regolare che denota il linguaggio $L(r)L(s)$
- r^* è un'espressione regolare che denota il linguaggio $L(r)^*$
- r^+ è un'espressione regolare che denota il linguaggio $L(r)^+$
- (r) è un'espressione regolare che denota il linguaggio $L(r)$

Le **definizioni regolari** introducono una **naming convention**. Dato un alfabeto Σ di simboli base, una definizione regolare è una sequenza di definizioni della forma:

$$\begin{aligned} d_1 &\to r_1 \\ d_2 &\to r_2 \\ &\dots \\ d_n &\to r_n \end{aligned}$$

Ogni d_i è un **nuovo simbolo**, non presente nell'alfabeto Σ, e diverso da ogni altro d_j con $j \neq i$. Ogni r_i è un'espressione regolare dell'alfabeto $\Sigma \cup \{d_1, d_2, \dots, d_{i-1}\}$.

Esempio 3.6. Il seguente esempio rappresenta una definizione regolare per gli identificatori in C:

$$\begin{aligned} \mathit{letter} &\to A \mid B \mid \dots \mid Z \mid a \mid b \mid \dots \mid z \\ \mathit{digit} &\to 0 \mid 1 \mid \dots \mid 9 \\ \mathit{id} &\to \mathit{letter}(\mathit{letter} \mid \mathit{digit})^* \end{aligned}$$

Esempio 3.7. Il seguente esempio rappresenta una definizione regolare per i numeri positivi (interi o decimali) che sono stringhe del tipo: `5280`, `0.0123`, `6.336E4`, `1.88E-2`:

$$\begin{aligned} \mathit{digit} &\to 0 \mid 1 \mid \dots \mid 9 \\ \mathit{digits} &\to \mathit{digit}\ \mathit{digit}^* \\ \mathit{optionalFraction} &\to .\mathit{digits} \mid \varepsilon \\ \mathit{optionalExponent} &\to (E(+ \mid - \mid \varepsilon)\mathit{digits}) \mid \varepsilon \\ \mathit{number} &\to \mathit{digits}\ \mathit{optionalFraction}\ \mathit{optionalExponent} \end{aligned}$$

3.5.5 Precedenza degli operatori

Gli operatori nelle espressioni regolari hanno la seguente precedenza:

- **Chiusura di Kleene** * e **chiusura positiva** $^+$ hanno la precedenza più alta e sono associativi a sinistra.
- **Concatenazione** rs è il secondo operatore per precedenza ed è associativo a sinistra.
- **Unione** $r \mid s$ è l'operatore con la precedenza più bassa ed è associativo a sinistra.

Esempio 3.8. La seguente espressione regolare:

$$(a) \mid ((b) * (c))$$

può essere sostituita con:

$$a \mid b^*c$$

Se due espressioni regolari r e s denotano lo stesso linguaggio, si dice che r e s sono **equivalenti** e si scrive $r = s$.

3.5.6 Regole algebriche per le espressioni regolari

Le espressioni regolari soddisfano le seguenti regole algebriche:

- $|$ è commutativo:
$$r \mid s = s \mid r$$
- $|$ è associativo:
$$r \mid (s \mid t) = (r \mid s) \mid t$$
- La concatenazione è associativa:
$$r(st) = (rs)t$$
- La concatenazione distribuisce su $|$:
$$r(s \mid t) = rs \mid rt$$
$$(s \mid t)r = sr \mid tr$$
- ε è l'identità per la concatenazione:
$$\varepsilon r = r\varepsilon = r$$
- ε è garantito in una chiusura:
$$r^* = (r \mid \varepsilon)^*$$
- La chiusura di Kleene è idempotente:
$$r^{**} = r^*$$

3.5.7 Notazione compatta

Per rendere più compatte le espressioni regolari, si possono utilizzare le seguenti abbreviazioni:

- $r^+ \rightarrow rr^*$
- $r? \rightarrow r \mid \varepsilon$
- $[r_1\, r_2 \ldots r_q] = r_1 \mid r_2 \mid \ldots \mid r_q$
- $[a - z] = a \mid b \mid \ldots \mid z$

3.6 Diagrammi di transizione

I **diagrammi di transizione** rappresentano espressioni regolari e definizioni regolari e sono definiti da:

- **Stati**: riassumono tutte le informazioni riguardo i caratteri compresi tra i due puntatori `lexemeBegin` e `forward`
- **Archi**: rappresentano le transizioni tra stati, etichettati con i caratteri che causano la transizione
- **Stato iniziale**: è lo stato in cui si trova il lexer prima di leggere qualsiasi carattere
- **Stati finali**: è un insieme di stati che indicano che è stato riconosciuto un token valido
- **Operatore ***: utilizzato per **ritrarre il puntatore** `forward`
- Sono diagrammi deterministici

3.6.1 Grammatica modello

Nelle sezioni successive utilizzeremo la seguente grammatica come modello per costruire i diagrammi di transizione:

- Grammatica:

$$
\begin{aligned}
\textit{stmt} &\rightarrow \texttt{if}\ \textit{expr}\ \texttt{then}\ \textit{stmt} \\
&\quad \mid \texttt{if}\ \textit{expr}\ \texttt{then}\ \textit{stmt}\ \texttt{else}\ \textit{stmt} \\
&\quad \mid \varepsilon \\
\textit{expr} &\rightarrow \texttt{term}\ \textit{relop}\ \texttt{term} \\
&\quad \mid \texttt{term} \\
\textit{term} &\rightarrow \texttt{id} \\
&\quad \mid \texttt{number}
\end{aligned}
$$

- Definizioni regolari:

$$
\begin{aligned}
\textit{digit} &\rightarrow \texttt{[0-9]} \\
\textit{digits} &\rightarrow \textit{digit}^{+} \\
\textit{number} &\rightarrow \textit{digits}\,(.\textit{digits})\texttt{?(E[+-]?)?[0-9]} \\
\textit{letter} &\rightarrow \texttt{[A-Za-z]} \\
\textit{id} &\rightarrow \textit{letter}(\textit{letter} \mid \textit{digit}) \\
\textit{if} &\rightarrow \texttt{if} \\
\textit{then} &\rightarrow \texttt{then} \\
\textit{else} &\rightarrow \texttt{else} \\
\textit{relop} &\rightarrow \texttt{<} \mid \texttt{>} \mid \texttt{<=} \mid \texttt{>=} \mid \texttt{<>} \mid \texttt{=} \\
\textit{ws} &\rightarrow (\texttt{blank} \mid \texttt{tab} \mid \texttt{newline})^{+}
\end{aligned}
$$

Il token *ws* rappresenta gli spazi bianchi ed è diverso dagli altri token perchè quando viene riconosciuto, il lexer non lo restituisce al parser, ma lo ignora e ricomincia l'analisi lessicale dal carattere successivo.

Da questa definizione si ricava la seguente tabella:

LEXEMES	TOKEN NAME	ATTRIBUTE VALUE
Any *ws*	–	–
`if`	**if**	–
`then`	**then**	–
`else`	**else**	–
Any *id*	**id**	Pointer to table entry
Any *number*	**number**	Pointer to table entry
`<`	**relop**	`LT`
`<=`	**relop**	`LE`
`=`	**relop**	`EQ`
`<>`	**relop**	`NE`
`>`	**relop**	`GT`
`>=`	**relop**	`GE`

Tabella 1: Tabella delle definizioni regolari

Esempio 3.9. Un esempio di diagramma di transizione per il token `relop` è il seguente:

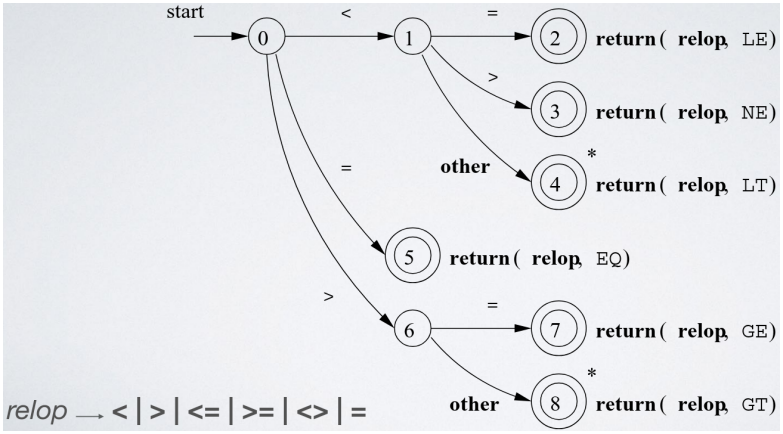


Figura 15: Esempio di diagramma di transizione per il token `relop`

Il codice di implementazione è del tipo:

```
TOKEN getRelop() {
  TOKEN retToken = new(RELOP);
  // repeat character processing until a return or failure
    occurs
```

```
  while (1) {
    // state is the current state of the transition diagram
    switch (state) {
      case 0:
        char c = nextChar();
        if (c == '<') state = 1;
        else if (c == '=') state = 5;
        else if (c == '>') state = 6;
        else fail(); // lexeme is not a relop
        break;

      case 1: //...
      //...
      case 8:
        retract();
        retToken.attribute = GT;
        return retToken;
    }
  }
}
```

Esempio 3.10. Un esempio di diagramma di transizione per il token `number` è il seguente:

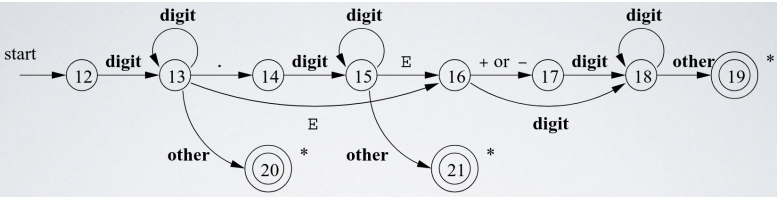


Figura 16: Esempio di diagramma di transizione per il token `number`

3.6.2 Stringhe riservate

Le stringhe riservate sono stringhe che corrispondono a token specifici, ad esempio le parole chiave del linguaggio. Per riconoscere gli identificatori si usa il seguente diagramma:

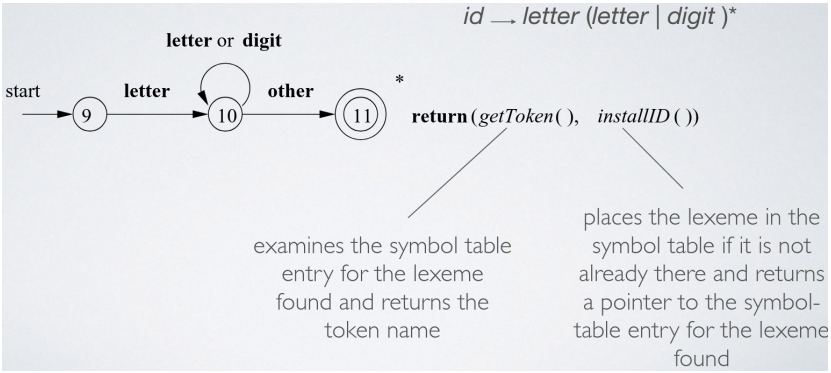


Figura 17: Esempio di diagramma di transizione per il token `id`

Per evitare che un identificatore utilizzi una parola chiave bisogna creare un diagramma di transizione per ogni parola chiave e dare **priorità** a questi lessemi quando ci sono più match possibili. Ad esempio, se il lexer riconosce la stringa `then`, deve riconoscerla come la parola chiave e non come un identificatore, quindi il diagramma di transizione per `then` è il seguente:

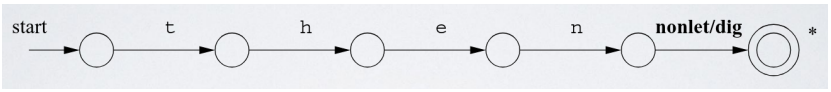


Figura 18: Esempio di diagramma di transizione per il token `then`

3.7 Architettura di un diagram-based lexer

Un diagram-based lexer è un lexer che utilizza i diagrammi di transizione per riconoscere i token e ha le seguenti proprietà:

- Ogni token ha un diagramma di transizione associato che viene tentato sequenzialmente. Successivamente la funzione `fail()` resetta il puntatore `forward` e tenta il diagramma di transizione del token successivo.
- Si eseguono i vari diagrammi di transizione in **parallelo**, dando in input il carattere successivo a tutti i diagrammi contemporaneamente permettendo a tutti di fare qualsiasi transizione richiesta. Alla fine si restituisce:
  - Il match più lungo, oppure
  - In caso di pareggio, il token con la priorità più alta
- Si combinano tutti i diagrammi di transizione in uno solo

Esempio 3.11. Un esempio di combinazione di diagrammi di transizione è il seguente:

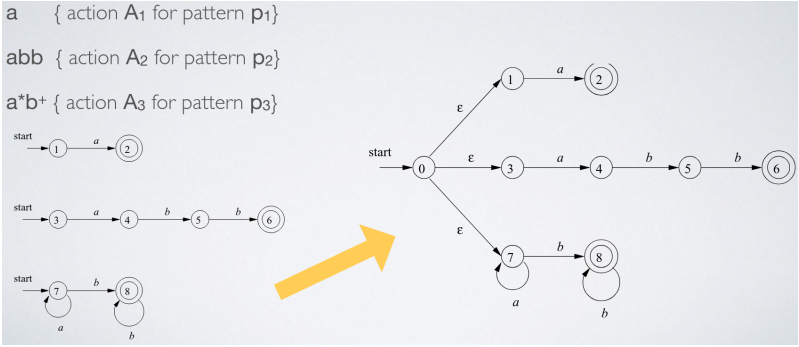


Figura 19: Esempio di combinazione di diagrammi di transizione

Esempio 3.12. Consideriamo il seguente diagramma che riconosce le espressioni regolari per i token:

- a
- abb
- a^*b^+

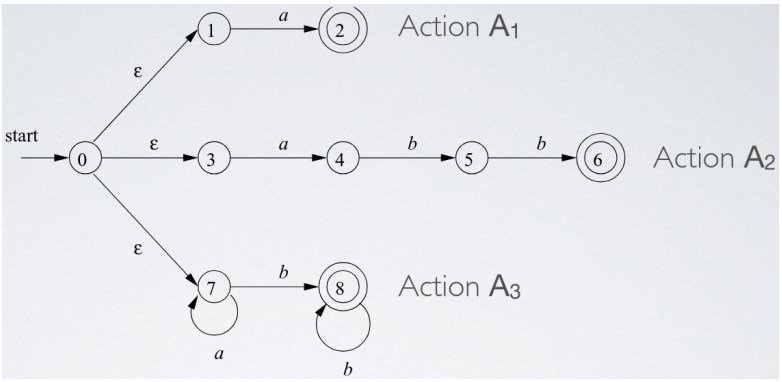


Figura 20: Esempio di combinazione di diagrammi di transizione per i token a, abb e a^*b^+

Se l'input è aabb, il lexer restituisce ad ogni passo l'insieme di stati in cui la simulazione del diagramma combinato si trova. L'esecuzione va avanti finchè l'insieme di stati è vuoto, cioè non ci sono più transizioni possibili con il carattere successivo, e a quel punto si torna indietro e si restituisce il token con il match più lungo:

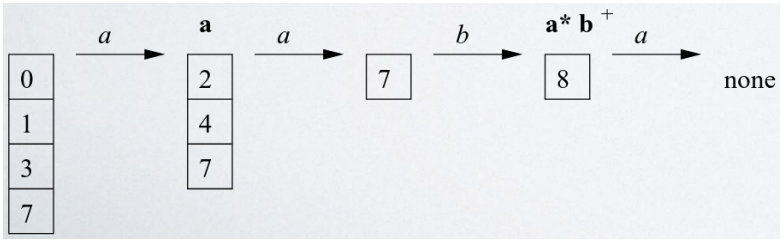


Figura 21: Esempio di esecuzione del diagramma combinato per l'input $aabb$

In questo caso, il lexer restituisce il token a^*b^+ con il lessema $aabb$ e procederà con l'analisi lessicale dei caratteri successivi.

3.7.1 Lex: Lexical Analyzer Generator

Lex[1] è un tool di generazione di analizzatori lessicali che permette di specificare un lexer utilizzando espressioni regolari. Gli input sono specificati nel linguaggio Lex.

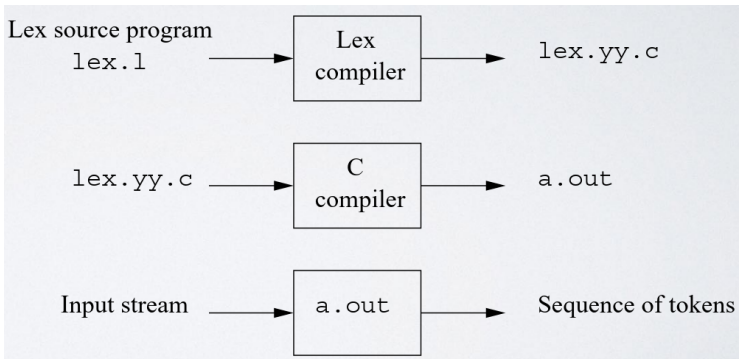


Figura 22: Esempio di generazione di Lex

3.7.2 Architettura di un lexical analyzer generator

Un lexical analyzer generator è un tool che prende in input una specifica del lexer e produce in output un programma che implementa il lexer. Il compito del lexical analyzer generator è:

- Tradurre espressioni regolari in NFA
- Tradurre NFA in DFA
- Minimizzare il DFA

[1] `https://thiagoh.github.io/lex/`



Figura 23: Esempio di architettura di un lexical analyzer generator

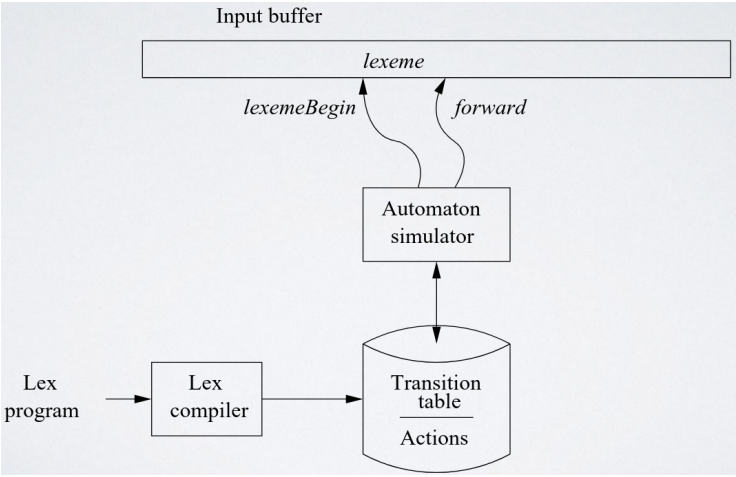


Figura 24: Diagramma di architettura di un lexical analyzer generator

3.7.3 Simulare un DFA

Per simulare un DFA, si utilizza un codice simile al seguente:

```
s = s_0;
c = nextChar();
while (c != eof) {
  s = move(s, c);
  c = nextChar();
}
if (s is in F) accept();
else reject();
```

dove:

- `move(s, c)` restituisce lo stato raggiunto partendo dallo stato `s` e leggendo il carattere `c`
- `nextChar()` restituisce il prossimo carattere dell'input

La complessità di questo algoritmo è $O(|x|)$ dove x è la stringa da analizzare.

3.7.4 Convertire un NFA in un DFA

Per convertire un NFA in un DFA, si utilizza l'algoritmo di **subset construction** che fa corrispondere ad ogni stato del DFA generato un insieme di stati dell'NFA

originale. Dopo aver letto l'input $a_1 a_2 \dots a_n$, il DFA si trova nello stato che corrisponde all'insieme di stati che l'NFA può raggiungere, partendo dallo stato iniziale, seguendo i percorsi etichettati $a_1 a_2 \dots a_n$. È possibile che il numero di stati del DFA generato sia esponenziale rispetto al numero di stati dell'NFA originale, ma in pratica per linguaggi reali il numero di stati del DFA è simile a quello dell'NFA.

Per convertire un NFA in un DFA, bisogna definire le seguenti funzioni:

- `ε-closure(s)`: restituisce l'insieme degli stati raggiungibili dallo stato s di un NFA seguendo solo transizioni ε.
- `ε-closure(T)`: restituisce l'insieme degli stati raggiungibili da alcuni stati s dell'insieme T di un NFA seguendo solo transizioni ε:

$$\cup_{s \in T} \varepsilon\text{-closure}(s)$$

```
push all states of T onto stack;
initialize ε—closure(T) to T;
while (stack not empty) {
    t = pop stack;
    for (each state u with an edge from t to u labeled ε) {
        if (u is not in ε—closure(T)) {
            add u to ε—closure(T);
            push u onto stack;
        }
    }
}
```

- `move(T, a)`: restituisce l'insieme degli stati raggiungibili da alcuni stati s dell'insieme T di un NFA seguendo solo transizioni etichettate a

L'algoritmo di subset construction è il seguente:

```
initially, ε—closure(s_0 ) is the only state in Dstates; and it is unmarked;
while (there is an unmarked state T in Dstates) {
    mark T;
    for (each input symbol a) {
        U = ε—closure(move(\textit{T}, \( a \)));
        if (U is not in Dstates) {
            add U as an unmarked state to Dstates;
        }
        add a transition from T to U on input a;
    }
}
```

Esempio 3.13. Consideriamo il seguente NFA che riconosce il linguaggio $L(N) = (a \mid b)^* abb$:

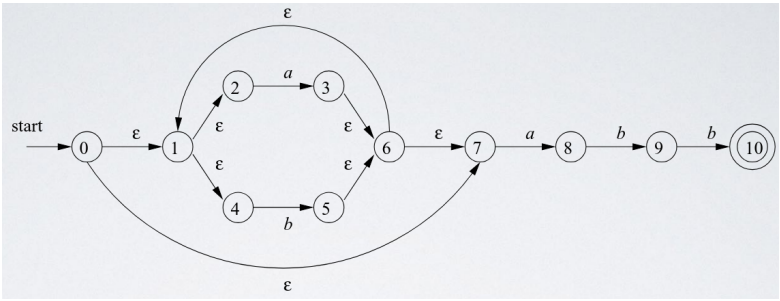


Figura 25: Esempio di NFA

Si trasformano gli stati dell'NFA in stati del DFA come mostrato nella seguente tabella:

NFA STATE	DFA STATE	a	b
$\{0, 1, 2, 4, 7\}$	A	B	C
$\{1, 2, 3, 4, 6, 7, 8\}$	B	B	D
$\{1, 2, 4, 5, 6, 7\}$	C	B	C
$\{1, 2, 4, 5, 6, 7, 9\}$	D	B	E
$\{1, 2, 4, 5, 6, 7, 10\}$	E	B	C

Tabella 2: Esempio di trasformazione da NFA a DFA

Il DFA risultante è il seguente:

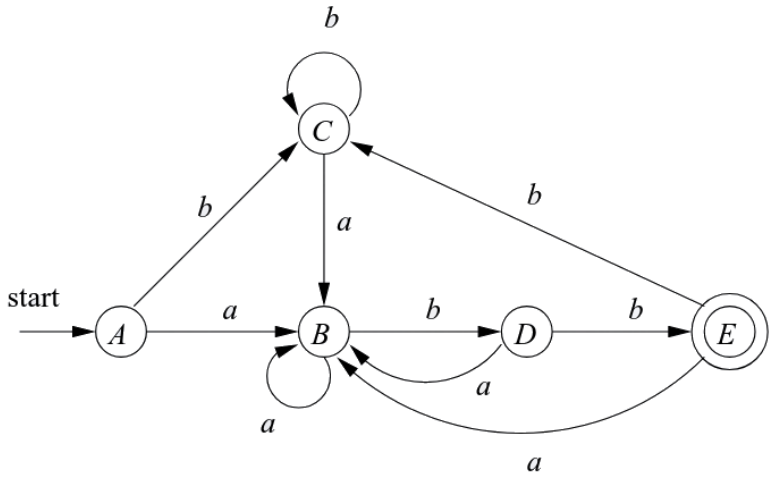


Figura 26: Esempio di DFA risultante dalla trasformazione da NFA a DFA

3.7.5 Simulare un NFA

Per simulare un NFA, si utilizza un codice simile al seguente:

```
S = ε-closure(\( s_0 \));
c = nextChar();
while (c != eof) {
    S = ε-closure(move(S, c));
    c = nextChar();
}
if (S ∩ F ≠ ∅) accept();
else reject();
```

La complessità di questo algoritmo è $O(|x| \cdot |r|)$ dove x è la stringa da analizzare e r è l'espressione regolare che rappresenta il linguaggio riconosciuto dall'NFA.

3.7.6 Trasformare espressioni regolari in NFA

Definizione 3.4 (Stati importanti)**.** Gli stati importanti di un NFA sono tutti quelli che hanno almeno una transizione in uscita che non sia una transizione ε, cioè se:

$$\texttt{move}(\{s\}, a) \neq \varnothing \quad \text{per qualche simbolo } a$$

allora lo stato s è uno stato importante.

Per trasformare un'espressione regolare in un NFA si seguono i seguenti passi:

1. **Aumentare** l'espressione regolare con un simbolo finale speciale, in questo caso #, che fa si che anche gli stati finali dell'NFA risultante siano stati importanti. Ad esempio, se l'espressione regolare è r, si trasforma in $r\#$.
2. Costruire un syntax tree per $r\#$

   Esempio 3.14. Prendiamo ad esempio l'espressione regolare

   $$(a \mid b)^* abb\#$$

   il syntax tree è il seguente:

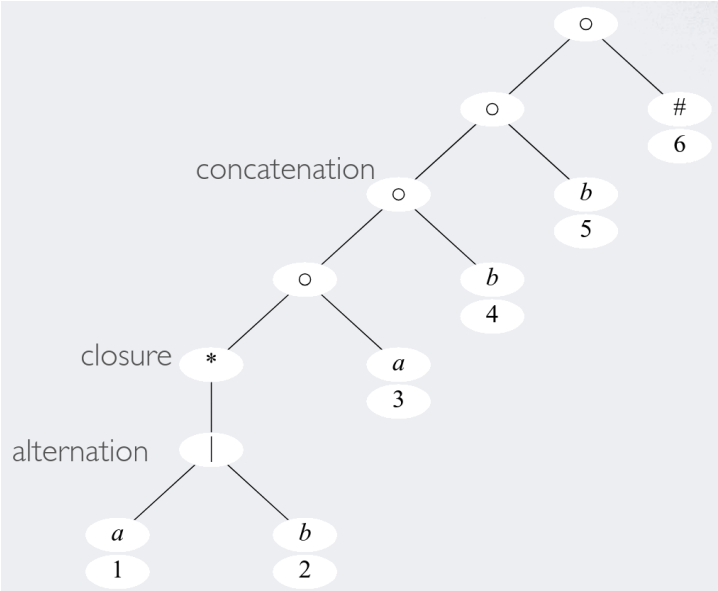


Figura 27: Esempio di syntax tree per l'espressione regolare (a | b)*abb#

Per ogni foglia diversa da ε si assegna un numero di posizione univoco.

3. Percorrere il syntax tree per costruire le funzioni:
   - `nullable(n)`: è `true` per un nodo n del syntax tree se la sottoespressione rappresentata da n contiene la stringa vuota ε nel suo linguaggio.
   - `firstpos(n)`: insieme di posizioni che possono matchare il primo simbolo di una stringa generata dal sottoalbero con radice n
   - `lastpos(n)`: insieme di posizioni che possono matchare l'ultimo simbolo di una stringa generata dal sottoalbero con radice n
   - Le funzioni precedenti si calcolano secondo la seguente tabella:

Node n	*nullable*(n)	*firstpos*(n)	*lastpos*(n)
Leaf ε	true	$\varnothing$	$\varnothing$
Leaf i	false	$\{i\}$	$\{i\}$
	*nullable*(c_1) or *nullable*(c_2)	*firstpos*(c_1) ∪ *firstpos*(c_2)	*lastpos*(c_1) ∪ *lastpos*(c_2)
	*nullable*(c_1) and *nullable*(c_2)	**if** *nullable*(c_1) **then** *firstpos*(c_1) ∪ *firstpos*(c_2) **else** *firstpos*(c_1)	**if** *nullable*(c_2) **then** *lastpos*(c_1) ∪ *lastpos*(c_2) **else** *lastpos*(c_2)
	true	*firstpos*(c_1)	*lastpos*(c_1)

Tabella 3: Tabella per il calcolo delle funzioni `nullable`, `firstpos` e `lastpos`

- `followpos(n)`: per una posizione p, è l'insieme di posizioni q in tutto il syntax tree tale che esiste una stringa $x = a_1 a_2 \dots a_n$ in $L((r)\#)$ tale che per qualche i, esiste un modo di spiegare l'appartenenza di x a $L((r)\#)$ facendo matchare a_i alla posizione p del syntax tree e a_{i+1} alla posizione q.

  Ci sono soltanto due modi in cui una posizione del syntax tree può essere seguita da un'altra posizione:

  (a) Se n è un nodo con operatore di concatenazione con figlio sinistro c_1 e figlio destro c_2, allora per ogni posizione i in `lastpos`(c_1) tutte le posizioni in `firstpos`(c_2) sono in `followpos`(i)

  (b) Se n è un nodo con operatore di chiusura di Kleene e i è una posizione in `lastpos`(n), allora tutte le posizioni in `firstpos`(n) sono in `followpos`(i)

Esempio 3.15. Considerando l'espressione regolare $(a \mid b)^* abb\#$ e il syntax tree mostrato nell'esempio precedente, si calcolano le funzioni `nullable`, `firstpos`, `lastpos` per ogni nodo del syntax tree:

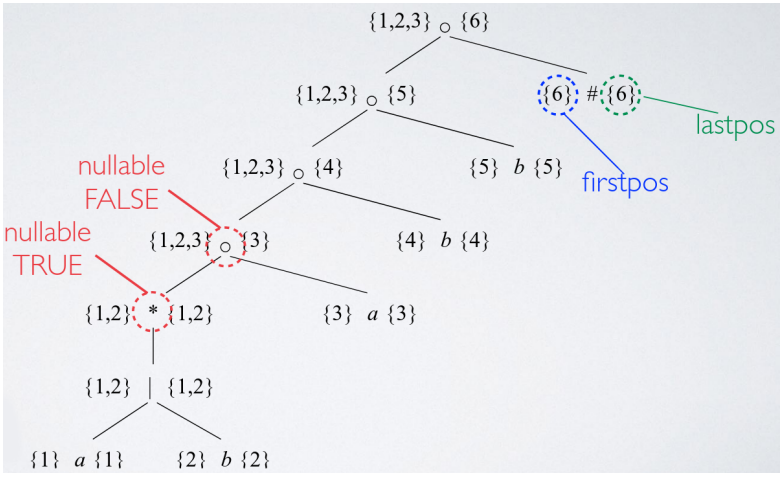
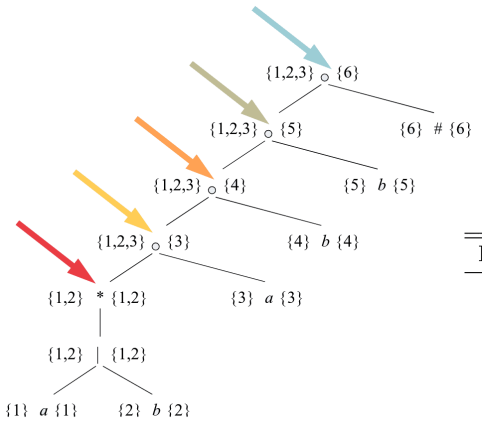


Figura 28: Esempio di calcolo delle funzioni `nullable`, `firstpos` e `lastpos` per il syntax tree dell'espressione regolare $(\mathtt{a} \mid \mathtt{b})^*\mathtt{abb}\#$

Successivamente, si calcolano le funzioni `followpos` per ogni nodo del syntax tree:



POSITION n	$followpos(n)$
1	$\{1, 2, 3\}$
2	$\{1, 2, 3\}$
3	$\{4\}$
4	$\{5\}$
5	$\{6\}$
6	$\emptyset$

Figura 29: Esempio di calcolo della funzione `followpos` per il syntax tree dell'espressione regolare $(\mathtt{a} \mid \mathtt{b})^*\mathtt{abb}\#$

L'algoritmo che prende in input un'espressione regolare $\mathtt{r}$ e costruisce un DFA che riconosce $\mathrm{L}(\mathtt{r})$ è il seguente:

initialize *Dstates* to contain only the unmarked state $firstpos(\mathtt{n}_0)$, where $\mathtt{n}_0$ is the root of the syntax tree T for $(\mathtt{r})\#$;
while (there is an unmarked state S in *Dstates*) {
  mark S;
  for (each input symbol a) {
    let U be the union of $followpos(p)$ for all positions p in S that correspond to a;
    if (U is not in *Dstates*) {

```
            add U as an unmarked state to Dstates;
        }
        Dtrans[S, a] = U;
    }
}
```

3.7.7 Minimizzare un DFA

Siccome possono esistere più DFA che riconoscono lo stesso linguaggio, è preferibile implementare un lexer con un DFA che abbia il numero minimo di stati possibile. Per ogni DFA esiste sempre un DFA **univoco equivalente** con il numero minimo di stati per ogni linguaggio regolare. Il DFA minimo può essere costruito da un qualsiasi DFA raggruppando insieme stati che sono **equivalenti**:

Definizione 3.5. La stringa x **distingue** lo stato s dallo stato t se esattamente uno degli stati raggiunti da s e t seguendo il percorso con etichetta x è uno stato di accettazione. Quindi lo stato s è distinguibile dallo stato t se esiste una stringa che li distingue.

Esempio 3.16. Nel seguente esempio, la stringa bb distingue gli stati A e B:

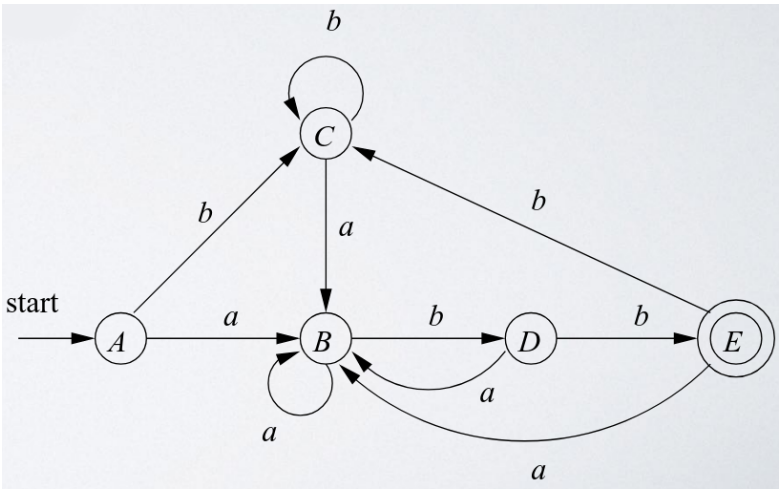


Figura 30: Esempio di DFA in cui la stringa bb distingue gli stati A e B

L'algoritmo per minimizzare un DFA segue i seguenti passi:

1. Si parte con una partizione iniziale Π con due gruppi, F e S − F, gli stati di accettazione e non di accettazione del DFA D.
2. Si costruisce una nuova partizione Π_{new}
3. Se $\Pi_{new} = \Pi$, allora $\Pi_{final} = \Pi$ e si continua con il passo successivo. Altrimenti, si ripete il passo 2 con Π_{new} al posto di Π.

4. Si sceglie uno stato in ogni gruppo di Π_{final} come rappresentante per quel gruppo. I rappresentanti saranno gli stati del DFA minimo $\mathrm{D}_{\mathrm{min}}$. Gli altri componenti di $\mathrm{D}_{\mathrm{min}}$ sono costruiti come segue:

   - Lo stato iniziale di $\mathrm{D}_{\mathrm{min}}$ è il rappresentante del gruppo che contiene lo stato iniziale di D
   - Gli stati finali di $\mathrm{D}_{\mathrm{min}}$ sono i rappresentanti di quei gruppi che contengono uno stato di accettazione di D
   - Sia s il rappresentante di un gruppo G di Π_{final}, e sia t lo stato raggiunto da s con una transizione su un input a in D. Sia r il rappresentante del gruppo H di t. Allora in $\mathrm{D}_{\mathrm{min}}$ c'è una transizione da s a r su input a.

```
1 initially, let Π_new = Π;
2 for (each group G of Π) {
3     partition G into subgroups such that two states s and t are in the same
        subgroup if and only if for all input symbols a, states s and t have
        transitions on a to states in the same group of Π;
4     \* at worst, a state will be in a subgroup by itself *\
5     replace G in Π_new by the set of all subgroups formed;
6 }
```

Esempio 3.17. Un esempio di partizioni per la minimizzazione di un DFA è il seguente:

- Initial partition: {A,B,C,D} and {E}
- {A,B,C} {D} {E}
- Final partition {A,C} {B} {D} {E}

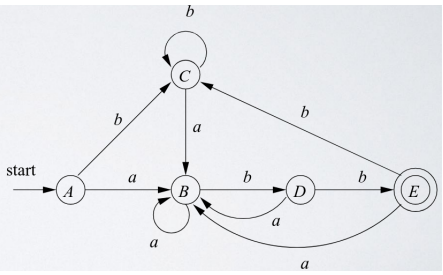


Figura 31: Esempio di partizioni per la minimizzazione di un DFA

4 Analisi sintattica (Parsing)

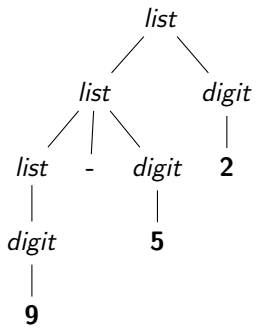
Il parsing è il processo di analisi sintattica che prende in input una sequenza di token e determina se la sequenza è sintatticamente corretta secondo la grammatica del linguaggio e costruisce un albero sintattico astratto (AST) che rappresenta la struttura gerarchica del programma. L'albero ha le seguenti caratteristiche:

- La **radice** dell'albero è etichettato con il simbolo di partenza della grammatica.
- Ogni **foglia** dell'albero è etichettato con un simbolo terminale (un token) o la stringa vuota ε.

- Ogni **nodo interno** è etichettato con un simbolo non terminale
- Se $A \to X_1 X_2 \dots X_n$ è una produzione, allora il nodo A ha n figli, etichettati con $X_1, X_2, \dots, X_n$ dove X_i è un terminale o non terminale (oppure ε.

Il processo per trovare un albero sintattico per una data stringa di terminali è chiamato **parsing** di quella stringa.

Esempio 4.1. L'albero sintattico per la stringa "**9 - 5 + 2**" secondo la grammatica dell'esempio precedente è il seguente:



La derivazione per questa stringa è la seguente:

$$\begin{aligned} \textit{list} &\to \textit{list} + \textit{digit} \\ &\to \textit{list} - \textit{digit} + \textit{digit} \\ &\to \textit{digit} - \textit{digit} + \textit{digit} \\ &\to \mathbf{9} - \textit{digit} + \textit{digit} \\ &\to \mathbf{9} - \mathbf{5} + \textit{digit} \\ &\to \mathbf{9} - \mathbf{5} + \mathbf{2} \end{aligned}$$

4.1 Definizione della sintassi

In questo corso verrà implementato un front-end per un compilatore:

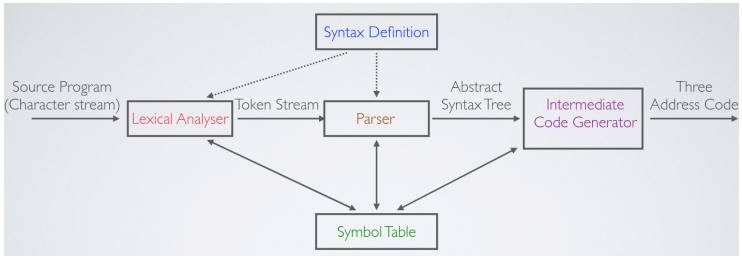


Figura 32: Front-end di un compilatore

La sintassi di un linguaggio di programmazione è definita attraverso le grammatiche context free. Una grammatica context free è composta da:

- Un insieme di **simboli terminali**, che rappresnetano simboli, numeri e parole chiave del linguaggio. Solitamente scritte in grassetto, ad esempio: **<**, **=**, **+**, **1**, **2**, **while**, ecc...
- Un insieme di **simboli non terminali**, che rappresentano categorie sintattiche come espressioni, istruzioni, blocchi, ecc... Solitamente scritte in corsivo, ad esempio: *expr*, *stmt*, *var*, ecc...
- Un insieme di **produzioni**, che definiscono come i simboli non terminali possono essere sostituiti da sequenze di simboli terminali e non terminali:

$$A \to \alpha \quad A \in V(\text{Non terminali}), \alpha \in (V \cup T)^*(\text{T terminali})$$

- Un simbolo non terminale di partenza

Esempio 4.2. L'esempio di una grammatica è il seguente:

- Simboli terminali: {**+**, **-**, **0**, **1**, **2**, **3**, **4**, **5**, **6**, **7**, **8**, **9**}
- Simboli non terminali: {*list*, *digit*}
- Simbolo di partenza: *list*
- Produzioni:

$$\begin{aligned} &\textit{list} \to \textit{list} + \textit{digit} \\ &\textit{list} \to \textit{list} - \textit{digit} \\ &\textit{list} \to \textit{digit} \\ &\textit{list} \to \mathbf{0} \mid \mathbf{1} \mid \mathbf{2} \mid \mathbf{3} \mid \mathbf{4} \mid \mathbf{5} \mid \mathbf{6} \mid \mathbf{7} \mid \mathbf{8} \mid \mathbf{9} \end{aligned}$$

Questa grammatica definisce le liste di numeri separati da operatori + o -.

4.1.1 Derivazioni

Data una grammatica context free si può determinare l'insieme di tutte le stringhe (sequenze di token) generate dalla grammatica utilizzando le **derivazioni**. Si parte dal simbolo di partenza e ad ogni passo si sostituisce un simbolo non terminale nella **forma sentenziale** (cioè una stringa che contiene sia simboli terminali che non terminali) con la parte destra di una produzione che ha quel simbolo non terminale.

Ci sono due tipi di derivazioni:

- **Derivazione sinistra**: ad ogni passo si sostituisce il simbolo non terminale più a sinistra. Ad esempio:

$$E \Rightarrow -E \Rightarrow -(E) \Rightarrow -(E + E) \Rightarrow -(\mathbf{id} + E) \Rightarrow -(\mathbf{id} + \mathbf{id})$$

- **Derivazione destra**: ad ogni passo si sostituisce il simbolo non terminale più a destra. Ad esempio:

$$E \Rightarrow -E \Rightarrow -(E) \Rightarrow -(E + E) \Rightarrow -(E + \mathbf{id}) \Rightarrow -(\mathbf{id} + \mathbf{id})$$

Teorema 4.1. Ogni albero sintattico è associato ad **una unica derivazione sinistra** e ad **una unica derivazione destra**, e viceversa.

4.2 Ambiguità

Una grammatica è ambigua se esiste una stringa che può essere generata da più di un albero sintattico. Siccome una stringa con più di un albero sintattico solitamente ha più di un significato, bisogna progettare grammatiche non ambigue per le applicazioni di compilazione, o utilizzare grammatiche ambigue con regole aggiuntive per risolvere le ambiguità.

Esempio 4.3. Considerando la seguente grammatica:

- Simboli terminali: {**+, -, 0, 1, 2, 3, 4, 5, 6, 7, 8, 9**}
- Simboli non terminali: {*string*}
- Simbolo di partenza: *string*
- Produzioni:

$$\begin{aligned} &\textit{string} \rightarrow \textit{string} + \textit{string} \\ &\textit{string} \rightarrow \textit{string} - \textit{string} \\ &\textit{string} \rightarrow \mathbf{0} \mid \mathbf{1} \mid \mathbf{2} \mid \mathbf{3} \mid \mathbf{4} \mid \mathbf{5} \mid \mathbf{6} \mid \mathbf{7} \mid \mathbf{8} \mid \mathbf{9} \end{aligned}$$

La stringa "**9 - 5 + 2**" può essere generata da due alberi sintattici diversi, che hanno significati diversi:

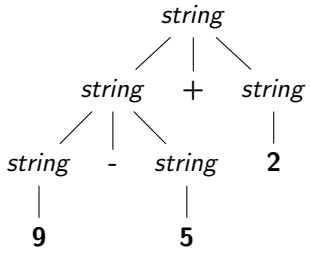


Figura 33: (9 - 5) + 2 = 6

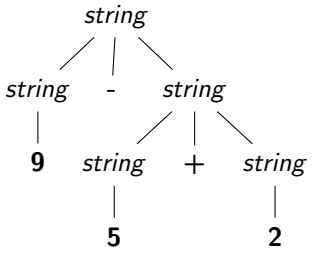


Figura 34: 9 - (5 + 2) = 2

L'ordine delle operazioni è diverso nei due alberi sintattici, e quindi la grammatica è ambigua.

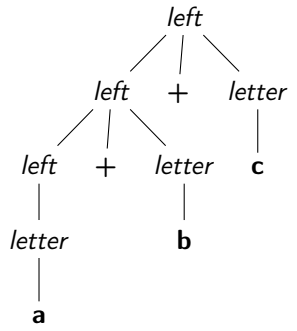
L'ambiguità può essere usata per semplificare la grammatica, ma poi bisogna crearne una equivalente non ambigua per usarla durante il parsing. È impossibile convertire automaticamente una grammatica ambigua in una equivalente non ambigua.

4.3 Associatività degli operatori

Nella maggior parte dei linguaggi di programmazione le quattro operazioni aritmetiche (+, -, *, /) sono **associate a sinistra**, cioè:

Esempio 4.4. La stringa `a + b + c` è interpretata come `(a + b) + c`

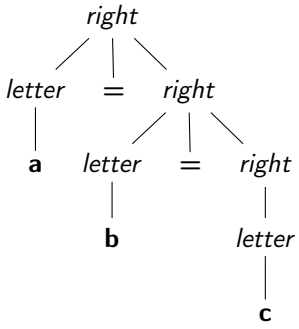
$$\mathtt{left} \rightarrow \mathtt{left} + \mathtt{letter} \mid \mathtt{letter}$$



Altre operazioni, come gli assegnamenti o l'esponenziazione sono invece **associate a destra**, cioè:

Esempio 4.5. La stringa `a = b = c` è interpretata come `a = (b = c)`

$$\mathtt{right} \rightarrow \mathtt{letter} = \mathtt{right} \mid \mathtt{letter}$$



4.4 Precedenza degli operatori

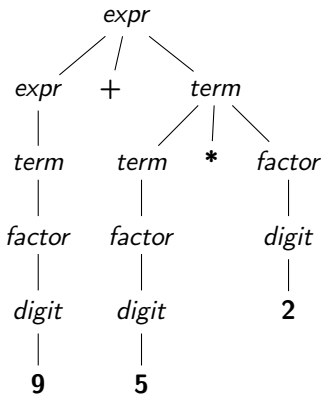
Bisogna introdurre regole che definiscono la precedenza relativa degli operatori quando più di un tipo di operatore è presente, ad esempio $*$ e $/$ hanno precedenza più alta di $+$ e $-$.

Esempio 4.6. La stringa `9 + 5 * 2` ha il significato di `9 + (5 * 2)`. Si può quindi costruire una grammatica per le operazioni aritmetiche che rispetta la

precedenza degli operatori:

$$
\begin{aligned}
\mathit{expr} &\rightarrow \mathit{expr} + \mathit{term} \mid \mathit{expr} - \mathit{term} \mid \mathit{term} \\
\mathit{term} &\rightarrow \mathit{term} * \mathit{factor} \mid \mathit{term} / \mathit{factor} \mid \mathit{factor} \\
\mathit{factor} &\rightarrow \mathit{digit} \mid (\mathit{expr}) \\
\mathit{digit} &\rightarrow \mathbf{0} \mid \mathbf{1} \mid \mathbf{2} \mid \mathbf{3} \mid \mathbf{4} \mid \mathbf{5} \mid \mathbf{6} \mid \mathbf{7} \mid \mathbf{8} \mid \mathbf{9}
\end{aligned}
$$

L'albero sintattico per la stringa `9 + 5 * 2` secondo questa grammatica è il seguente:



E ha come significato `9 + (5 * 2)` = 19.

4.5 Sintassi degli statements

Uno statement è una unità sintattica che rappresenta un'azione da eseguire, ad esempio `if`, `while`, `for`, ecc... .

Esempio 4.7. Un esempio di grammatica per gli statements è il seguente:

$$
\begin{aligned}
\mathit{stmt} &\rightarrow \mathbf{id} = \mathit{expr}; \\
&\mid \textbf{if } \mathit{expr} \textbf{ then } \mathit{stmt} \\
&\mid \textbf{if } \mathit{expr} \textbf{ then } \mathit{stmt} \textbf{ else} \mathit{stmt} \\
&\mid \textbf{while } \mathit{expr} \textbf{ do } \mathit{stmt} \\
&\mid \textbf{do } \mathit{stmt} \textbf{ while } \mathit{expr}; \\
&\mid \{ \mathit{stmt} \} \\
\\
\mathit{stmts} &\rightarrow \mathit{stmts}\ \mathit{stmt} \\
&\mid \varepsilon
\end{aligned}
$$

4.6 Errori

Siccome un compilatore può tradurre soltanto programmi validi, è importante che sia in grado di riconoscere quando un programma non è valido. Ci sono diversi tipi di errori:

- **Errori lessicali**: include errori di spelling di identificatori, keywords o operatori e sono rilevati durante l'analisi lessicale.
- **Errori sintattici**: include errori di sintassi, come ad esempio punti e virgola fuori posto o parentesi graffe extra o mancanti. In questo caso, le singole unità lessicali sono corrette, ma non possono essere analizzate sintatticamente, e gli errori sono rilevati durante l'analisi sintattica.
- **Errori semantici**: include errori di tipo e sono rilevati dal Type Checker.
- **Errori di correttezza**: sono errori che non possono essere rilevati dai compilatori, ma devono essere individuati tramite testing.

4.6.1 Gestione degli errori

In generale l'error handler di un parser dovrebbe:

- **Riportare** la presenza di errori in modo chiaro e accurato
- Cercare di **riprendere** l'analisi sintattica dopo un errore, in modo da rilevare ulteriori errori in un singolo passaggio
- **Non dovrebbe rallentare** il processo di parsing di programmi validi

Ci sono diversi modi per gestire gli errori sintattici:

- **Panic mode**: È il metodo più semplice e popolare. Quando viene rilevato un errore:
  - Ignora tutti i token finchè non viene trovato un token con un ruolo valido detto **synchronizing token**
  - Continua l'analisi sintattica a partire da quel token

  I synchronizing token sono tipicamente i terminatori di espressioni o di statements.

  Esempio 4.8. Considerando il seguente codice:

  ```
  (1 ++ 2) + 3
  ```

  Una volta che viene trovato il token $+$ in più, il parser ignora tutti i token fino all'intero successivo e continua l'analisi sintattica a partire da quel token.

  Il problema della panic mode è che può portare a **cascading errors**, cioè errori che sono causati da un errore precedente, ma che non sono errori reali. Solitamente fixare il primo errore rilevato potrebbe risolvere anche tutti gli errori successivi.

- **Error productions**: Consiste nell'aggiungere alla grammatica le produzioni per gli errori più comuni, perchè se vengono fatti spesso forse è meglio aggiungerli come feature del linguaggio.

  Esempio 4.9. Ad esempio per permettere di scrivere `5x` al posto di `5 * x` si aggiunge la seguente produzione alla grammatica:

  $$\texttt{expr} \rightarrow \texttt{expr}\ \ \texttt{expr} \mid \ldots$$

  Questo metodo spesso non è la soluzione migliore e ha lo svantaggio di complicare la grammatica.

- **Correzione automatica locale o globale**: L'idea è quella di cercare un programma corretto **"vicino"** e sostituire le parti errate del programma con le parti corrette di un altro programma. Questo metodo è molto complesso da implementare, rallenta molto il parsing e il programma corretto "vicino" non è necessariamente quello che l'utente intendeva scrivere.

  Questo metodo era molto utile in passato, quando i compilatori erano così lenti che era preferibile correggere automaticamente gli errori piuttosto che farli correggere manualmente dagli utenti e ricompilare il programma più volte.

4.7 Top-down parsing (Recursive descent)

Il Top-down parsing costruisce l'albero sintattico partendo dalla radice annotata con il simbolo di partenza e applicando due passi:

1. Seleziona un nodo annotato con un simbolo non terminale A partendo da **sinistra**
2. Seleziona una produzione per A e genera tanti figli di A quanti sono i simboli a destra della produzione

Questa procedura finisce quando tutti i nodi foglia sono annotati con simboli terminali, oppure quando non è possibile applicare nessuna produzione a un nodo annotato con un simbolo non terminale. In quest'ultimo caso, la stringa di input non è generata dalla grammatica.
La selezione di una produzione per un non terminale può comportare **backtracking**, cioè provare una produzione e se non funziona tornare indietro e provare un'altra produzione.

Esempio 4.10. Il codice per parsare un non terminale A è il seguente:

```
1 void A() {
2     Choose an A−production, A → X1X2 . . . Xk;
3       for (i = 1 to k) {
4           if (Xi is not terminal) {
5             call procedure Xi();
6         } else if (Xi equals the current input symbol a) {
7             advance the input to the next symbol;
8         } else {
```

```
9           report an error; // or backtrack and try another production
10      }
11  }
12 }
```

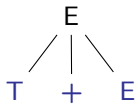
Esempio 4.11. Consideriamo il seguente esempio di backtracking con la grammatica:

$$E \to T + E \mid T$$
$$T \to \textbf{id} * T \mid \textbf{id} \mid (E)$$

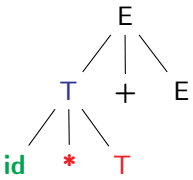
Vogliamo parsare la stringa `id + id`. Il parser esegue i seguenti passi:

1. Inizia dalla radice dell'albero che contiene il simbolo non terminale E

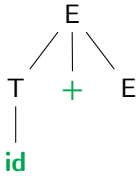
2. Usa la produzione $E \to T + E$



3. Usa la produzione $T \to \textbf{id} * T$, e si ha che **id** corrisponde al primo token di input, ma $*$ non corrisponde al secondo token

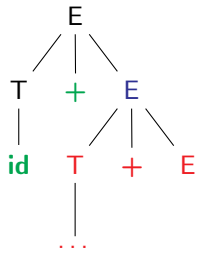


4. **Backtracking**: usa l'altra produzione per T: $T \to$ **id**; il token **id** corrisponde e anche il token $+$

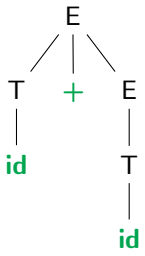


5. Bisogna scegliere una produzione per il secondo E

- Se si sceglie $E \to T + E$ si fallisce, quindi si fa backtracking



- Se si sceglie la produzione $E \to T$ si sceglie anche una produzione per T e si ottiene l'albero sintattico con successo



4.7.1 Predictive parsing

Il predictive parsing è un metodo di top-down parsing che utilizza un simbolo di lookahead per decidere quale produzione applicare. il **Recursive descent parsing** è un metodo di top-down parsing in cui viene utilizzato un insieme di procedure ricorsive per processare l'input, in cui **ogni non terminale** ha una **procedura ricorsiva** che si occupa di fare il parsing di quel non terminale. In generale la selezione di una produzione per un non terminale può comportare un approccio trial-and-error, ma il predictive parsing **evita il backtracking** utilizzando un **simbolo di lookahead** per determinare in modo univoco quale produzione applicare.

Definizione 4.1 (Simbolo di lookahead)**.** Il simbolo di lookahead è un simbolo terminale che viene utilizzato per decidere quale produzione applicare durante il parsing. Ad esempio, se il simbolo di lookahead è `for`, allora si sa che bisogna applicare la produzione $\texttt{stmt} \to \texttt{for}(\texttt{optexpr}; \texttt{optexpr}; \texttt{optexpr})\texttt{stmt}$.

Quando un non terminale ha più produzioni, ogni produzione viene implementata in un ramo di una dichiarazione di selezione basata sulle informazioni del simbolo di lookahead. Il predictive parsing è una forma speciale di recursive descent parsing in cui si utilizza un simbolo di lookahead per determinare in modo univoco le operazioni di parsing.

Esempio 4.12. Considerando la seguente grammatica:

$$\begin{aligned} \text{stmt} &\rightarrow \text{expr};\\ &\quad |\text{if}(\text{expr})\text{stmt}\\ &\quad |\text{for}(\text{optexpr};\text{optexpr};\text{optexpr})\text{stmt}\\ &\quad |\text{other} \end{aligned}$$

$$\begin{aligned} \text{optexpr} &\rightarrow \text{expr}\\ &\quad |\varepsilon \end{aligned}$$

Il predictive parser per questa grammatica è il seguente:

```
void stmt() {
  switch (lookahead) {
    case expr:
      match(expr);
      match(';');
      break;
    case if:
      match(if);
      match('(');
      match(expr);
      match(')');
      stmt();
      break;
    case for:
      match(for);
      match('(');
      optexpr();
      match(';');
      optexpr();
      match(';');
      optexpr();
      match(')');
      stmt();
      break;
    case other:
      match(other);
      break;
    default:
      report("Syntax error");
  }
}

void optexpr() {
  if (lookahead == expr) {
    match(expr);
  }
}

void match(terminal t) {
  if (lookahead == t) {
    lookahead = nextTerminal();
  } else {
    report("Syntax error");
  }
```

```
45 }
```

4.7.2 Funzione first

La funzione $\text{FIRST}(\alpha)$ è l'insieme dei terminali che appaiono come primi simboli di una o più stringhe generate da α.

Esempio 4.13. Considerando la grammatica dell'esempio precedente si ha:

$$\text{FIRST}(\text{stmt}) = \{\text{expr}, \text{if}, \text{for}, \text{other}\}$$
$$\text{FIRST}(\text{optexpr}) = \{\text{expr}, \varepsilon\}$$

4.7.3 Ricorsione sinistra immediata

Una produzione del tipo:

$$A \to A\alpha \mid \beta$$

è detta **ricorsiva sinistra** perchè il primo simbolo del corpo della produzione è lo stesso del non terminale alla testa della produzione. La ricorsione sinistra può causare un **loop infinito** nel Top-down parsing, perchè il parser siccome sceglie le produzioni partendo da sinistra applicherebbe la produzione senza mai arrivare a una produzione che non è ricorsiva sinistra.

Per risolvere questo problema, si possono eliminare le produzioni ricorsive sinistra riscrivendo la grammatica in modo da utilizzare produzioni ricorsive a destra. Ad esempio:

$$A \to A\alpha \mid \beta$$
$$\Downarrow$$
$$A \to \beta R$$
$$R \to \alpha R \mid \varepsilon$$

Definizione 4.2 (Eliminazione della ricorsione sinistra immediata)**.** Data una produzione ricorsiva sinistra della forma:

$$A \to A\alpha_1 \mid A\alpha_2 \mid \ldots \mid A\alpha_m \mid \beta_1 \mid \beta_2 \mid \ldots \mid \beta_n$$

Dove $\beta_1, \ldots, \beta_n$ sono produzioni che non iniziano con A, allora una grammatica equivalente senza ricorsione sinistra è la seguente:

$$A \to \beta_1 R \mid \ldots \mid \beta_n R$$
$$R \to \alpha_1 R \mid \ldots \mid \alpha_m R \mid \varepsilon$$

Se la grammatica ha ricorsione sinistra indiretta, cioè se esiste una sequenza di più di una produzione che porta a una produzione ricorsiva sinistra, esiste un algoritmo per eliminare la ricorsione sinistra indiretta, ma è complesso e non verrà trattato in questo corso.

4.8 Predictive parsing

Il predictive parsing è un metodo di top-down parsing che utilizza un simbolo di lookahead per decidere quale produzione applicare. Questo metodo funziona soltanto su una forma particolare di grammatiche, dette $\mathrm{LL}(k)$ **grammars**. Il predictive parser è completamente deterministico, cioè non ha bisogno di backtracking perchè ad ogni passo soltanto una produzione è applicabile.

I criteri per la selezione di una produzione sono i seguenti:

- Se la parte a destra della produzione **inizia con un token**, allora la produzione verrà usata solo se il token è uguale al simbolo di lookahead
- Se la parte a destra della produzione **inizia con un non terminale**, allora la produzione verrà usata solo se il simbolo di lookahead può essere generato da quel non terminale

4.8.1 Grammatiche LL(k)

I predictive parser accettano soltanto grammatiche $\mathrm{LL}(k)$ che si chiamano così perchè:

- La prima L sta per **Left-to-right scanning** dell'input
- La seconda L sta per **Leftmost derivation** dell'albero sintattico
- k è il numero di token di lookahead utilizzati per decidere quale produzione applicare

Le grammatiche $\mathrm{LL}(k)$ sono un sottoinsieme delle grammatiche context free:

$$\mathrm{LL}(1) \subset \mathrm{LL}(2) \subset \ldots \subset \mathrm{CF}$$

Nella pratica utilizziamo soltanto grammatiche $\mathrm{LL}(1)$ perchè sono più semplici da implementare.

4.8.2 Left factoring

Il left factoring è una tecnica che permette di trasformare una grammatica in una forma adatta al predictive parsing, eliminando le ambiguità che possono essere causate da prefissi comuni nelle produzioni di un non terminale.

Definizione 4.3 (Left factoring)**.** Per ogni non terminale A, cercare il più lungo prefisso non vuoto comune a due o più produzioni di A. Se esiste un prefisso comune, estrarlo in una nuova produzione e sostituire il prefisso comune con la nuova produzione nelle produzioni originali.

$$A \to \alpha\beta_1 \mid \alpha\beta_2 \mid \ldots \mid \alpha\beta_n \mid \gamma$$

viene trasformata in:

$$A \to \alpha A' \mid \gamma$$
$$A' \to \beta_1 \mid \beta_2 \mid \ldots \mid \beta_n$$

Esempio 4.14. Eseguiamo il left factoring sulla seguente grammatica:

$$E \to T \mid T + E$$
$$T \to \mathtt{int} \mid \mathtt{int} * T \mid (E)$$

La produzione $E \to T \mid T + E$ ha un prefisso comune T, quindi si estrae e lo stesso vale per la produzione $T \to \mathtt{int} \mid \mathtt{int} * T$ che ha un prefisso comune $\mathtt{int}$:

$$E \to T\ X$$
$$X \to + E \mid \varepsilon$$
$$T \to \mathtt{int}\ Y \mid (E)$$
$$Y \to * T \mid \varepsilon$$

4.8.3 Tabella di parsing

Definiamo le funzioni FIRST e FOLLOW che ci permettono di costruire la tabella.

Definizione 4.4. La funzione FIRST(X) è definita come:

$$\text{FIRST}(X) = \{t \mid X \to^* t\alpha\} \cup \{\varepsilon \mid X \to^* \varepsilon\}$$

cioè l'insieme dei terminali che appaiono come primi simboli di una o più stringhe generate da X. ε appartiene a FIRST(X) in due casi:

- Se $X \to \varepsilon$
- Se $X \to A_1 \dots A_n$ e $\varepsilon \in \text{FIRST}(A_i)$ per ogni $1 \leqslant i \leqslant n$

Il FIRST di un terminale è il terminale stesso:

$$\text{FIRST}(t) = \{t\}$$

e di conseguenza si ha che:

$\text{FIRST}(\alpha) \subseteq \text{FIRST}(X)$ se $X \to A_1 \dots A_n \alpha \wedge \varepsilon \in \text{FIRST}(A_i) \quad \forall i\ .\ 1 \leqslant i \leqslant n$

Esempio 4.15. Consideriamo la seguente grammatica:

$$E \to T\ X$$
$$T \to \mathtt{int}\ Y \mid (E)$$
$$X \to + E \mid \varepsilon$$
$$Y \to * T \mid \varepsilon$$

Calcoliamo il FIRST di ogni simbolo:

- $\text{FIRST}(+) = \{+\}$ ecc... per ogni terminale
- $\text{FIRST}(\text{E}) = \text{FIRST}(\text{T})$
- $\text{FIRST}(\text{T}) = \{\texttt{int}, (\}$
- $\text{FIRST}(\text{X}) = \{+, \varepsilon\}$
- $\text{FIRST}(\text{Y}) = \{*, \varepsilon\}$

Definizione 4.5. La funzione $\text{FOLLOW}(\text{X})$ è definita come:

$$\text{FOLLOW}(\text{X}) = \{\text{t} \mid \text{S} \rightarrow^* \beta\text{X}\text{t}\delta\}$$

cioè l'insieme dei terminali che appaiono immediatamente a destra di X in una stringa generata da S. Inoltre:

- Se $\text{X} \rightarrow \text{AB}$ allora

$$\text{FIRST}(\text{B}) \subseteq \text{FOLLOW}(\text{A}) \wedge \text{FOLLOW}(\text{X}) \subseteq \text{FOLLOW}(\text{B})$$

- Se $\text{B} \rightarrow^* \varepsilon$ allora

$$\text{FOLLOW}(\text{X}) \subseteq \text{FOLLOW}(\text{A})$$

Solitamente si utilizza un simbolo speciale \$ per rappresentare la fine dell'input, e si ha che $\$ \in \text{FOLLOW}(\text{S})$ dove S è il simbolo di partenza.

Esempio 4.16. Considerando la stessa grammatica dell'esempio precedente, calcoliamo il FOLLOW di ogni simbolo:

- Per il simbolo E si ha:
  - $\text{FOLLOW}(\text{X}) \subseteq \text{FOLLOW}(\text{E})$
  - $\text{FOLLOW}(\text{E}) \subseteq \text{FOLLOW}(\text{X})$

  Quindi $\text{FOLLOW}(\text{E}) = \text{FOLLOW}(\text{X}) = \{), \$\}$
- Per il simbolo T si ha:
  - $\text{FOLLOW}(\text{E}) \subseteq \text{FOLLOW}(\text{T})$
  - $\text{FIRST}(\text{Y}) \subseteq \text{FOLLOW}(\text{T})$
  - $\text{FIRST}(\text{T}) \subseteq \text{FOLLOW}(\text{Y})$

  Quindi $\text{FOLLOW}(\text{T}) = \text{FOLLOW}(\text{Y}) = \{+,), \$\}$

- Per il simbolo (si ha:

$$\text{FOLLOW}(() = \text{FIRST}(\text{E}) = \text{FIRST}(\text{T}) = \{\texttt{int}, (\}$$

- Per il simbolo) si ha:

$$\text{FOLLOW}()) = \text{FOLLOW}(\text{T}) = \{+,), \$\}$$

- Per il simbolo $+$ si ha:

$$\text{FOLLOW}(+) = \text{FIRST}(\text{E}) = \text{FIRST}(\text{T}) = \{\texttt{int}, (\}$$

- Per il simbolo $*$ si ha:

$$\text{FOLLOW}(*) = \text{FIRST}(\text{T}) = \{\texttt{int}, (\}$$

- Per il simbolo $\texttt{int}$ si ha:

$$\text{FOLLOW}(\texttt{int}) = \text{FIRST}(\text{Y}) = \{*, +,), \$\}$$

Definizione 4.6 (Tabella di parsing)**.** Sia A un simbolo non terminale con una produzione $A \to \alpha$ e un token t. L'entrata della tabella $T[A, t] = \alpha$ in due casi:

1. Se $\alpha \to^* t\beta$, cioè se α può generare una stringa con primo simbolo t:

$$t \in \text{FIRST}(\alpha) \implies T[A, t] = \alpha$$

2. Se $t \notin \text{FIRST}(\alpha)$, ma $\alpha \to^* \varepsilon$ e $S \to^* \beta A t \delta$, cioè quando A non può generare una stringa con primo simbolo t, ma A può generare la stringa vuota e t può apparire immediatamente a destra di A:

$$t \notin \text{FIRST}(\alpha) \wedge \varepsilon \in \text{FIRST}(\alpha) \wedge t \in \text{FOLLOW}(A) \implies T[A, t] = \alpha$$

Per costruire una tabella di parsing T per la grammatica G si eseguono i seguenti passi:

1. Per ogni produzione $A \to \alpha$ della grammatica:
   (a) Per ogni terminale $t \in \text{FIRST}(\alpha)$, inserire α nella tabella $T[A, t]$
   (b) Se $\varepsilon \in \text{FIRST}(\alpha)$, allora per ogni terminale $t \in \text{FOLLOW}(A)$, inserire α nella tabella $T[A, t]$
   (c) Se $\varepsilon \in \text{FIRST}(\alpha)$ e $\$ \in \text{FOLLOW}(A)$, inserire α nella tabella $T[A, \$]$

lo pseudocodice è il seguente:

```
for each production A → α {
    for each terminal t ∈ FIRST(α) {
```

```
        T[A, t] = α
    }

    if ε ∈ FIRST(α) {
        for each terminal t ∈ FOLLOW(A) {
            T[A, t] = α
        }
        if $ ∈ FOLLOW(A) {
            T[A, $] = α
        }
    }
}
```

Esempio 4.17. Un esempio di tabella di parsing per la grammatica:

$$
\begin{aligned}
&E \to T\ X \\
&T \to \mathtt{int}\ Y \mid (E) \\
&X \to + E \mid \varepsilon \\
&Y \to * T \mid \varepsilon
\end{aligned}
$$

è il seguente:

	(	)	+	*	int	$
E	T X				T X	
T		(E)			int Y	
X		ε	+ E			ε
Y		ε	ε	* T		ε

4.8.4 Grammatiche LL(1)

Le condizioni per cui una grammatica **non** è più $\mathrm{LL}(1)$ sono le seguenti:

1. Se una qualsiasi cella della tabella di parsing contiene più di una produzione
2. Se la grammatica non è left factored
3. Se la grammatica è ricorsiva sinistra
4. Se la grammatica è ambigua

Quindi definiamo quando una grammatica è $\mathrm{LL}(1)$:

Definizione 4.7. Una grammatica G è $\mathrm{LL}(1)$ se non è ricorsiva sinistra e per ogni insieme di produzioni $A \to \alpha_1 \mid \alpha_2 \mid \ldots \mid \alpha_n$ si ha che per il simbolo non terminale A vale:

- $\mathrm{FIRST}(\alpha_i) \cap \mathrm{FIRST}(\alpha_j) = \varnothing$ per ogni $i \neq j$
- Se $\alpha_i \Rightarrow^* \varepsilon$ allora:

- $\alpha_j \not\Rightarrow^* \varepsilon$ per ogni $j \neq i$ e
- $\text{FIRST}(\alpha_j) \cap \text{FOLLOW}(A) = \varnothing$)

4.8.5 Predictive parser

I predictive parser si costruiscono usando grammatiche LL(1), cioè grammatiche in cui per ogni non terminale e token di lookahead c'è solo una produzione applicabile. Il parser può essere quindi specificato usando una tabella bidimensionale chiamata **tabella di parsing**, in cui:

- Una dimensione rappresenta il simbolo **non terminale corrente** da espandere
- Una dimensione rappresenta il token successivo

Ogni cella della tabella contiene la produzione da applicare oppure se è vuota rappresenta un errore sintattico. Il predictive parser può essere implementato in modo non ricorsivo gestendo manualmente uno stack per tenere traccia dei simboli non terminali da espandere.

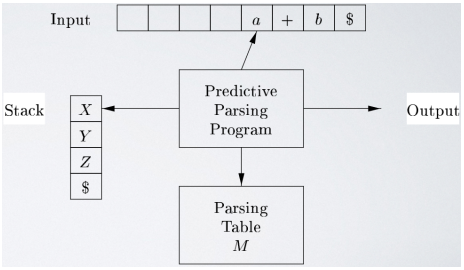


Figura 35: Modello di un predictive parser non ricorsivo

Lo stato iniziale del parser è costituito da uno stack che contiene $[S, \$]$ e da un buffer di input che contiene la stringa da parsare seguita da \$.

L'algoritmo per il predictive parsing è il seguente:

```
let a be the first symbol of w;
let X be the top stack symbol;
while ( X ≠ $ ) { /* stack is not empty */
    if ( X = a ) pop the stack and let a be the next symbol of w;
    else if ( X is a terminal ) error();
    else if ( M[X, a] is an error entry ) error();
    else if ( M[X, a] = X → Y1Y2 ··· Yk ) {
        output the production X → Y1Y2 ··· Yk;
        pop the stack;
        push Yk, Yk−1, ..., Y1 onto the stack, with Y1 on top;
    }
    let X be the top stack symbol;
}
```

Esempio 4.18. Consideriamo la seguente grammatica:

$$\begin{aligned}
&\mathrm{E} \to \mathrm{TE}' \\
&\mathrm{E}' \to +\mathrm{TE}' \mid \varepsilon \\
&\mathrm{T} \to \mathrm{FT}' \\
&\mathrm{T}' \to *\mathrm{FT}' \mid \varepsilon \\
&\mathrm{F} \to (\mathrm{E}) \mid \mathtt{id}
\end{aligned}$$

Costruiamo la tabella di parsing per questa grammatica e calcoliamo il FIRST e il FOLLOW di ogni simbolo:

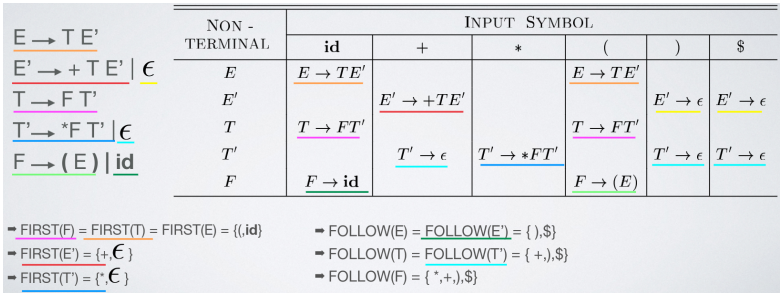


Figura 36: Tabella di parsing per la grammatica dell'esempio

Ora utilizzando questa tabella di parsing vogliamo parsare la stringa:

```
id + id * id
```

I passi di parsing sono i seguenti:

MATCHED	STACK	INPUT	ACTION
	$E\$$	$\mathbf{id} + \mathbf{id} * \mathbf{id}\$$	
	$TE'\$$	$\mathbf{id} + \mathbf{id} * \mathbf{id}\$$	output $E \to TE'$
	$FT'E'\$$	$\mathbf{id} + \mathbf{id} * \mathbf{id}\$$	output $T \to FT'$
	$\mathbf{id}\ T'E'\$$	$\mathbf{id} + \mathbf{id} * \mathbf{id}\$$	output $F \to \mathbf{id}$
$\mathbf{id}$	$T'E'\$$	$+\ \mathbf{id} * \mathbf{id}\$$	match $\mathbf{id}$
$\mathbf{id}$	$E'\$$	$+\ \mathbf{id} * \mathbf{id}\$$	output $T' \to \epsilon$
$\mathbf{id}$	$+\ TE'\$$	$+\ \mathbf{id} * \mathbf{id}\$$	output $E' \to +\ TE'$
$\mathbf{id} +$	$TE'\$$	$\mathbf{id} * \mathbf{id}\$$	match $+$
$\mathbf{id} +$	$FT'E'\$$	$\mathbf{id} * \mathbf{id}\$$	output $T \to FT'$
$\mathbf{id} +$	$\mathbf{id}\ T'E'\$$	$\mathbf{id} * \mathbf{id}\$$	output $F \to \mathbf{id}$
$\mathbf{id} + \mathbf{id}$	$T'E'\$$	$* \mathbf{id}\$$	match $\mathbf{id}$
$\mathbf{id} + \mathbf{id}$	$* FT'E'\$$	$* \mathbf{id}\$$	output $T' \to * FT'$
$\mathbf{id} + \mathbf{id} *$	$FT'E'\$$	$\mathbf{id}\$$	match $*$
$\mathbf{id} + \mathbf{id} *$	$\mathbf{id}\ T'E'\$$	$\mathbf{id}\$$	output $F \to \mathbf{id}$
$\mathbf{id} + \mathbf{id} * \mathbf{id}$	$T'E'\$$	$\$$	match $\mathbf{id}$
$\mathbf{id} + \mathbf{id} * \mathbf{id}$	$E'\$$	$\$$	output $T' \to \epsilon$
$\mathbf{id} + \mathbf{id} * \mathbf{id}$	$\$$	$\$$	output $E' \to \epsilon$

Tabella 4: Passi di parsing per la stringa `id + id * id`

Da questo si può ottenere la sequenza di derivazioni per ottenere la stringa di input:

$$\mathtt{E} \Rightarrow \mathtt{TE'} \Rightarrow \mathtt{FT'E'} \Rightarrow \mathtt{idT'E'} \Rightarrow \mathtt{idE'} \Rightarrow \mathtt{id\ +TE'} \Rightarrow \mathtt{id\ +FT'E'} \Rightarrow$$
$$\mathtt{id\ +\ idT'E'} \Rightarrow \mathtt{id\ +\ id\ *FT'E'} \Rightarrow \mathtt{id\ +\ id\ *\ idT'E'}$$
$$\Rightarrow \mathtt{id\ +\ id\ *\ idE'} \Rightarrow \mathtt{id\ +\ id\ *\ id}$$

4.9 Bottom-up parsing

Il bottom-up parsing è un metodo di parsing più generale rispetto al top down ed è quello più utilizzato perchè non richiede grammatiche left factored. L'idea di base è quella di ridurre le stringhe al simbolo di partenza **invertendo le produzioni** (le produzioni inverse si dicono **riduzioni**).

Esempio 4.19. Consideriamo la grammatica:

$$\mathtt{E} \to \mathtt{T} + \mathtt{E} \mid \mathtt{T}$$
$$\mathtt{T} \to \mathtt{int} \mid \mathtt{int\ *\ T} \mid (\mathtt{E})$$

Un esempio di riduzioni per la stringa `int + int * int` è il seguente:

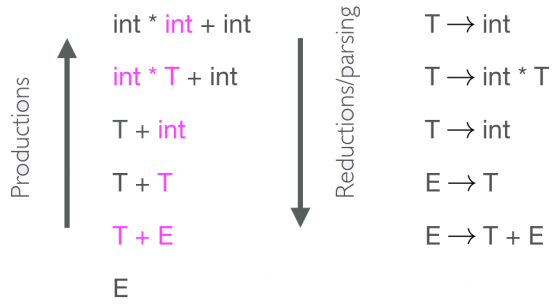


Figura 37: Esempio di riduzioni per la stringa `int + int * int`

4.9.1 Shift-reduce parsing

Un bottom-up parser traccia una **rightmost derivation in reverse** e questo implica che la parte a destra rispetto a dove il parser si trova è composta da soli terminali:

Teorema 4.2. Sia $\alpha\beta\omega$ un passo di un bottom-up parsing. Ipotizzando che la produzione successiva sia $A \to \beta$, allora ω è una stringa di soli terminali.

Nel bottom-up parsing si utilizza un carattere per indicare la posizione del parser: $\cdot$, ad esempio:

$$\alpha X \cdot \omega$$

Ad ogni passo di parsing una specifica sottostringa che matcha la parte destra di una produzione è **sostituita dal non terminale** a sinistra della stessa produzione. Bisogna capire **quando** e **quale produzione** applicare ad ogni passo e per farlo si introducono due azioni:

- **Shift**: si sposta un **terminale** a sinistra del $\cdot$ nella sottostringa che è stata processata, ad esempio:

$$ABC \cdot xyz \Rightarrow ABCx \cdot yz$$

- **Reduce**: si applica una **riduzione** (produzione inversa) alla sottostringa più a destra della stringa processata (a sinistra del $\cdot$) che matcha la parte destra di una produzione, ad esempio data la produzione $A \to xy$ si ha:

$$Cbxy \cdot ijk \Rightarrow CbA \cdot ijk$$

Esempio 4.20. Un esempio di applicazione delle azioni di shift e reduce per la stringa `int * int + int` della grammatica dell'esempio precedente è il seguente:

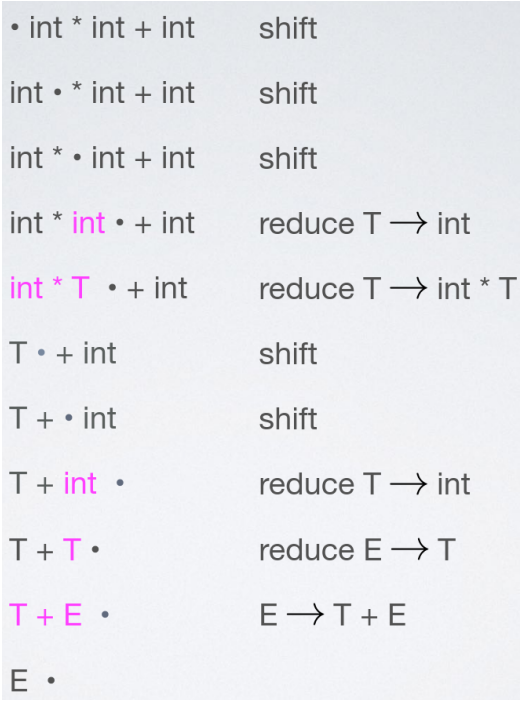


Figura 38: Esempio di applicazione delle azioni di shift e reduce per la stringa `int * int + int`

Per implementare un shift-reduce parser si utilizza uno stack dove la cima rappresenta il ·, cioè il punto in cui il parser si trova.

- Lo shift pusha un terminale sullo stack
- La reduce
  - Poppa 0 o più simboli dallo stack (produzione rhs, right hand side)
  - Pusha un non terminale sullo stack (produzione lhs, left hand side)

4.9.2 Scegliere le azioni di shift e reduce

La parte più difficile del bottom-up parsing è capire quando applicare uno shift e quando applicare una reduce.

Esempio 4.21. Consideriamo la grammatica:

$$\mathrm{E} \to \mathrm{T} \mid \mathrm{T} + \mathrm{E}$$
$$\mathrm{T} \to \texttt{int} \mid \texttt{int * T} \mid (\mathrm{E})$$

e il seguente passo di parsing della stringa `int * int + int`:

$$\texttt{int} \cdot \texttt{ * int + int}$$

In questo caso si potrebbe ridurre usando la produzione $T \to \texttt{int}$ e ottenere:

$$T \cdot \texttt{* int + int}$$

ma questo **è sbagliato** perchè da questo stato non si può più ridurre fino al simbolo iniziale E.

Si vuole quindi applicare la reduce **soltanto se il risultato si può ridurre al simbolo iniziale** (handle), altrimenti si applica lo shift.

4.9.3 Riconoscere gli handle

Definizione 4.8 (Handle)**.** Un **handle** è una sottostringa che matcha la parte destra di una produzione, e la cui riduzione rappresenta un passo lungo la reverse di una derivazione destra, cioè che ammette ulteriori riduzioni fino ad arrivare al simbolo iniziale.
Ad esempio, data la derivazione destra:

$$S \Rightarrow^* \alpha A \omega \Rightarrow \alpha\beta\omega$$

e la produzione $A \to \beta$, allora la sottostringa β è un handle di $\alpha\beta\omega$:

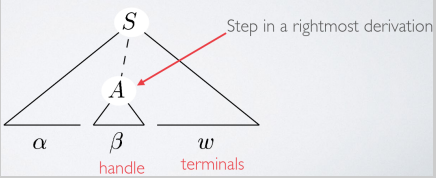


Figura 39: Esempio di handle

Teorema 4.3. Se la grammatica non è ambigua, allora ogni forma sentenziale destra di una grammatica ha esattamente un handle.

Non esistono algoritmi efficienti per riconoscere gli handle, ma esistono delle buone euristiche per riconoscerli e su **alcuni tipi** di grammatiche le euristiche riconoscono **sempre** gli handle.

In un bottom-up parser, durante una scansione da sinistra a destra della stringa di input, il parser esegue zero o più shift fino a quando è pronto per ridurre una stringa di simboli della grammatica sulla cima dello stack. A quel punto, riduce al non terminale a sinistra della produzione appropriata. **L'handle sarà eventualmente sempre in cima allo stack, mai all'interno** e si hanno due casi:

1. Caso 1: Si ha una derivazione del tipo:

$$S \Rightarrow_r^* m\alpha Az \Rightarrow_r m\alpha\beta Byz \Rightarrow_r m\alpha\beta\gamma yz$$

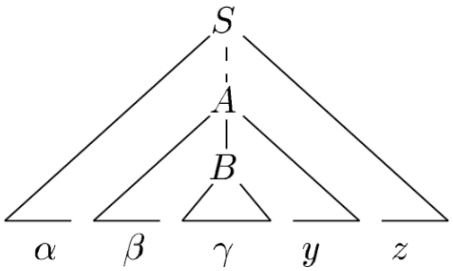


Figura 40: Albero della derivazione per il caso 1

Consideriamo un parser nella configurazione:

$$\alpha\beta\gamma \cdot yz$$

In questo caso, si riduce l'handle γ a B e si ottiene:

$$\alpha\beta B \cdot yz$$

Ora il parser esegue lo shift di y e si ottiene:

$$\alpha\beta By \cdot z$$

Con l'handle βBy in cima allo stack si può ridurre ad A.

2. Caso 2: Si ha una derivazione del tipo:

$$S \Rightarrow_r^* m\alpha BxAz \Rightarrow_r m\alpha Bxyz \Rightarrow_r m\alpha\gamma xyz$$

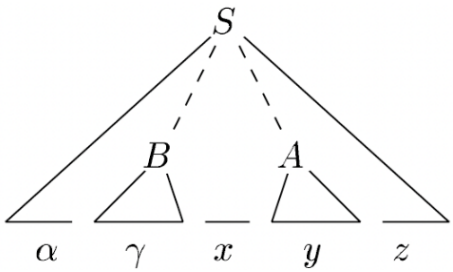


Figura 41: Albero della derivazione per il caso 2

Consideriamo un parser nella configurazione:

$$\alpha\gamma \cdot xyz$$

L'handle γ è in cima allo stack, quindi si riduce a B e si ottiene:

$$\alpha B \cdot xyz$$

Ora il parser esegue lo shift di x e Y per avere l'handle y in cima allo stack, e si ottiene:

$$\alpha Bxy \cdot z$$

In entrambi i casi, dopo aver effettuato una riduzione, il parser ha dovuto effettuare uno o più shift per portare l'handle in cima allo stack e non ha mai dovuto cercare nello stack per trovare l'handle.

Ad ogni passo il parser vede soltanto la cima dello stack e non tutto l'input e quindi si ha che in un parser nello stato:

$$\alpha \cdot \omega$$

α è detto **prefisso accettabile** perchè è il prefisso di un handle e **non supera mai la fine di un handle**. Finchè sono presenti prefissi accettabili nello stack, il parser non sono stati trovati errori di parsing. **Per ogni grammatica l'insieme di prefissi accettabili è un linguaggio regolare**, quindi si può costruire un automa a stati finiti per riconoscere i prefissi accettabili.

4.9.4 LR parsing

Il tipo più diffuso di parser bottom-up è basato su un concetto chiamato LR(k) parsing, dove:

- La L sta per **Left-to-right scanning** dell'input
- La R sta per **Rightmost derivation in reverse** dell'albero sintattico
- k è il numero di token di lookahead utilizzati per decidere quale produzione applicare

I parser LR sono table-driven e sono il metodo di parsing bottom-up più generale conosciuto che non richiede backtracking, e possono essere implementati in modo efficiente. Un parser LR può rilevare un errore sintattico non appena è possibile farlo durante una scansione da sinistra a destra dell'input. La classe di grammatiche che possono essere parsate usando i metodi LR è un sottoinsieme proprio della classe di grammatiche che possono essere parsate con i metodi LL.

Lo svantaggio principale del metodo LR è che è troppo complesso costruire un parser LR a mano per una tipica grammatica di un linguaggio di programmazione, quindi è necessario un tool specializzato, un generatore di parser LR.

Definizione 4.9 (Item)**.** Un item è qualsiasi produzione con un punto $\cdot$ in qualche posizione della parte destra della produzione. Ad esempio, la produzione $A \to XYZ$ ha 4 items:

1. $A \to \cdot XYZ$
2. $A \to X \cdot YZ$
3. $A \to XY \cdot Z$
4. $A \to XYZ\cdot$

Mentre la produzione $A \to \varepsilon$ ha soltanto un item:

1. $A \to \cdot$

Esempio 4.22. Data la produzione T $\rightarrow$ (E), l'item:

$$T \rightarrow (E\cdot)$$

ci dice che fin'ora il parser ha visto (E e si aspetta di vedere). La produzione verrà ridotta al shift successivo.

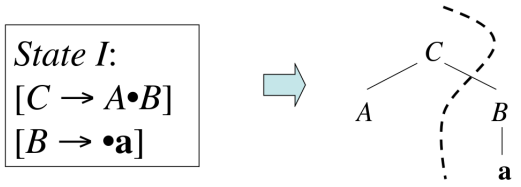
Per riconoscere i prefissi accettabili bisogna riconoscere una sequenza di parti destre parziali di produzioni dove ogni parte destra parziale si può eventualmente ridurre ad una parte del suffisso mancante del suo predecessore.

4.9.5 Collezione canonica di una grammatica LR(0)

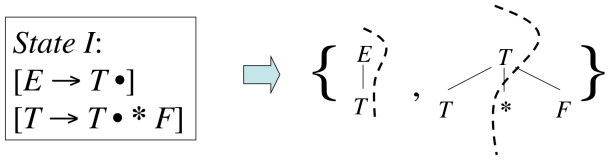
Definizione 4.10. La **collezione canonica** di una grammatica LR(k) è l'insieme di insiemi di item LR(k) (stati) che forniscono una base per costruire un automa a stati utilizzto per fare decisioni di parsing.

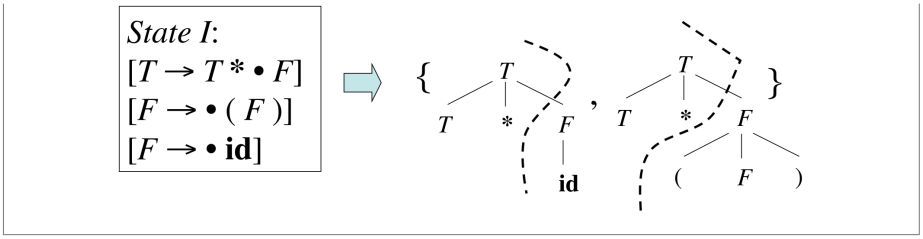
Esempio 4.23. Alcuni esempi di stati di varie grammatiche LR(0) sono i seguenti:

1. Gli stati sono insiemi di item



2. Gli stati possono rappresentare diversi alberi alternativi contemporaneamente





Per costruire la collezione canonica di una grammatica LR(0) bisogna definire:

- Data la grammatica G la grammatica G′ con un nuovo simbolo iniziale S' e una sola produzione $S' \to S$ dove S è il simbolo iniziale di G
- La funzione `CLOSURE`:
  Se I è un insieme di item per una grammatica G, allora `CLOSURE(I)` è l'insieme di item costruiti da I applicando le seguenti regole:
  1. Inizialmente aggiungere ogni item in I a `CLOSURE(I)`
  2. Se $A \to \alpha \cdot B\beta$ è in `CLOSURE(I)` e $B \to \gamma$ è una produzione, allora aggiungere l'item $B \to \cdot\gamma$ a `CLOSURE(I)` se non è già presente, e ripetere questo processo finchè non è più possibile aggiungere item a `CLOSURE(I)`.

Esempio 4.24. Un esempio di applicazione della funzione `CLOSURE` è il seguente:

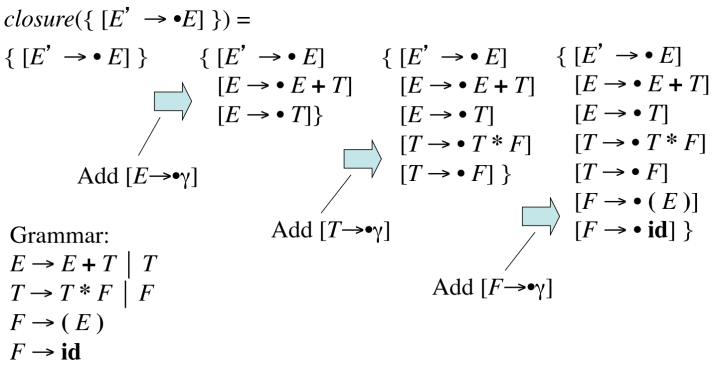


Figura 42: Esempio di applicazione della funzione `CLOSURE`

- La funzione `GOTO`:
  Prende in input lo stato corrente I e un simbolo della grammatica X ed è definita come la **closure dell'insieme di tutti gli item** $A \to \alpha X \cdot \beta$ tale che $A \to \alpha \cdot X\beta$ è in I.

  La funzione `GOTO` ritorna lo stato aggiornato spostando il punto $\cdot$ oltre il simbolo X e scartando gli item che non hanno X dopo il punto, e applicando la funzione `CLOSURE` al risultato.

Esempio 4.25. Un esempio di applicazione della funzione `GOTO` è il seguente:

Suppose $I = \{ [E' \rightarrow E \bullet], [E \rightarrow E \bullet + T] \}$

Then $goto(I,+) = closure(\{[E \rightarrow E + \bullet T]\}) = \{ [E \rightarrow E + \bullet T]$
$[T \rightarrow \bullet T * F]$
$[T \rightarrow \bullet F]$
$[F \rightarrow \bullet (E)]$
$[F \rightarrow \bullet \mathbf{id}] \}$

Grammar:
$E \rightarrow E + T \mid T$
$T \rightarrow T * F \mid F$
$F \rightarrow (E)$
$F \rightarrow \mathbf{id}$

Figura 43: Esempio di applicazione della funzione `GOTO`

Esempio 4.26. Un altro esempio di applicazione della funzione `GOTO` è il seguente:

Suppose $I =$
$\{ [E' \rightarrow \bullet E]$
$[E \rightarrow \bullet E + T]$
$[E \rightarrow \bullet T]$
$[T \rightarrow \bullet T * F]$
$[T \rightarrow \bullet F]$
$[F \rightarrow \bullet (E)]$
$[F \rightarrow \bullet \mathbf{id}] \}$

Then $goto(I, E)$
$= closure(\{[E' \rightarrow E \bullet], [E \rightarrow E \bullet + T]\})$
$= \{ [E' \rightarrow E \bullet], [E \rightarrow E \bullet + T] \}$

Grammar:
$E \rightarrow E + T \mid T$
$T \rightarrow T * F \mid F$
$F \rightarrow (E)$
$F \rightarrow \mathbf{id}$

Figura 44: Esempio di applicazione della funzione `GOTO`

L'algoritmo per calcolare la collezione canonica è il seguente:

```
1  void items(G′) {
2      C = { CLOSURE({[S′ → ·S]}) };
3      do {
4          for each set of items I ∈ C {
5              for each grammar symbol X {
6                  if GOTO(I, X) is not empty and not in C {
7                      add GOTO(I, X) to C;
8                  }
9              }
10         }
11     } while no new set of items is added to C on a round;
12 }
```

4.9.6 DFA per la decisione di shift e reduce

Il risultato delle funzioni `CLOSURE` e `GOTO` è un DFA in cui:

- **Stati**: sono insiemi di item della collezione canonica $\mathrm{LR}(0)$
- **Transizioni**: sono determinate dalla funzione `GOTO` e rappresentano i possibili spostamenti del punto $\cdot$ in un item
- **Stato iniziale**: è lo stato che contiene l'item $S' \to \cdot S$ dove S' è il nuovo simbolo iniziale della grammatica G'
- Tutti gli stati sono stati finali che riconoscono i prefissi accettabili

Considerando una stringa di simboli della grammatica che porta l'automa $\mathrm{LR}(0)$ dallo stato iniziale 0 ad uno stato j, allora si effettua:

- **Shift** sull'input successivo a se lo stato j ha una transizione su un simbolo di input a
- **Reduce** altrimenti.

Gli item nello stato j determinano quale produzione utilizzare per la riduzione.

Esempio 4.27. Data la grammatica aumentata:

$$
\begin{aligned}
E' &\to E \\
E &\to E + T \mid T \\
T &\to T * F \mid F \\
F &\to (E) \mid \texttt{id}
\end{aligned}
$$

e la stringa di input `id * id`, il DFA per la decisione di shift e reduce è il seguente:

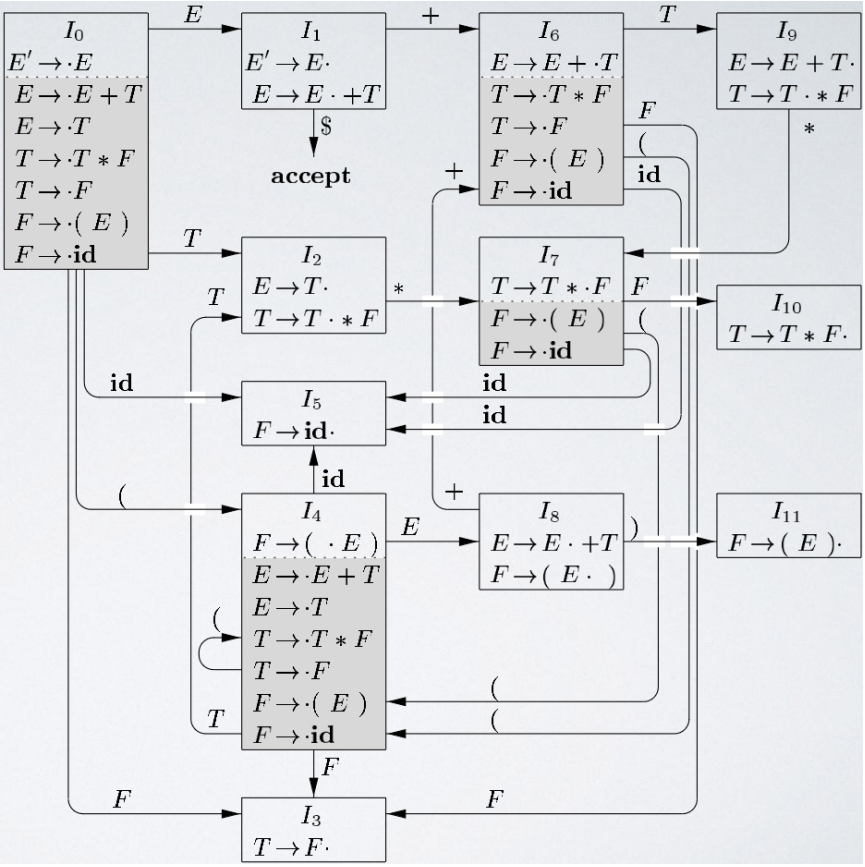


Figura 45: Esempio di DFA per la decisione di shift e reduce

L'esecuzione di questo DFA per la stringa di input `id * id` è la seguente:

Line	Stack	Symbols	Input	Action
(1)	0	\$	**id** $*$ **id** \$	shift to 5
(2)	0 5	\$ **id**	$*$ **id** \$	reduce by $F \to$ **id**
(3)	0 3	\$ F	$*$ **id** \$	reduce by $T \to F$
(4)	0 2	\$ T	$*$ **id** \$	shift to 7
(5)	0 2 7	\$ $T *$	**id** \$	shift to 5
(6)	0 2 7 5	\$ $T *$ **id**	\$	reduce by $F \to$ **id**
(7)	0 2 7 10	\$ $T * F$	\$	reduce by $T \to T * F$
(8)	0 2	\$ T	\$	reduce by $E \to T$
(9)	0 1	\$ E	\$	accept

Tabella 5: Esecuzione del DFA per la stringa di input `id * id`

4.9.7 Algoritmo di parsing LR

L'input di questo algoritmo è:

- Una stringa w

- Una tabella LR con le funzioni `ACTION` e `GOTO` per una grammatica G

L'algoritmo è il seguente:

```
let a be the first symbol of w$;
while(1) { /* repeat forever */
    let s be the state on top of the stack;
    if ( ACTION[s, a] = shift t ) {
        push t onto the stack;
        let a be the next input symbol;
    } else if ( ACTION[s, a] = reduce A → β ) {
        pop |β| symbols off the stack;
        let state t now be on top of the stack;
        push GOTO[t, A] onto the stack;
        output the production A → β;
    } else if ( ACTION[s, a] = accept ) break; /* parsing is done */
    else call error−recovery routine;
}
```

Lo stato del parser è detto **configurazione** ed è rappresentato dalla coppia stack e input imanente:

$$(s_0 \dots s_m,\ a_i \dots a_n\$)$$

che rappresenta la forma sentenziale destra $X_1 \dots X_m a_i \dots a_n$.

Se l'azione da fare nello stato s_m con il token di lookahead a_i (`ACTION`$[s_m, a_i]$) è uguale a:

- **shift** s allora si pusha s e si avanza l'input:

$$(s_0 \dots s_m s,\ a_{i+1} \dots a_n\$)$$

- **reduce** $A \to \beta$ e `GOTO`$[s_{m-r}, A] = s$ con $r = |\beta|$, cioè con s_{m-r} che è lo stato che si trova sotto i simboli β che devono essere ridotti, allora si poppano r simboli dallo stack e si pusha s:

$$(s_0 \dots s_{m-r} s,\ a_i \dots a_n\$)$$

- **accept** allora si accetta la stringa di input e si termina il parsing
- **error** allora si cerca di fare error recovery

4.9.8 Simple LR parser (SLR)

Il SLR parsing si basa ul fatto che il DFA per un parser LR(0) riconosce i prefissi accettabili. Data una grammatica G si costruisce la grammatica aumentata G′ da cui si costruisce la collezione canonica C e la funzione `GOTO` si costruisce la tabella di parsing SLR con le funzioni `ACTION` e `GOTO` usando il seguente algoritmo:

```
let C = {I_0, I_1, ..., I_n} be the set of LR(0) states for G′;
let the initial state 0 be the I_i holding [S′ → ·S];

for each state I_i ∈ C {
    for each item in I_i {
```

```
        if item is [S′ → S·] {
            ACTION[i, $] = accept;
        } else  if item is [A → α · aβ] and GOTO(Ii, a) = Ij {
            ACTION[i, a] = shift j;
        } else if item is [A → α·] and A ≠ S′ {
            for each terminal a ∈ FOLLOW(A) {
                ACTION[i, a] = reduce A → α;
            }
        }
    }
    for each nonterminal A {
        if GOTO(Ii, A) = Ij {
            GOTO[i, A] = j;
        }
    }
}
```

Nella tabella di parsing:

- si rappresenta uno shift sullo stack dello stato si
- rj rappresenta una reduce usando la produzione j
- acc rappresenta l'accettazione della stringa di input
- una cella vuota rappresenta un errore sintattico

Esempio 4.28. Un esempio di costruzione della tabella di parsing SLR per la il DFA seguente:

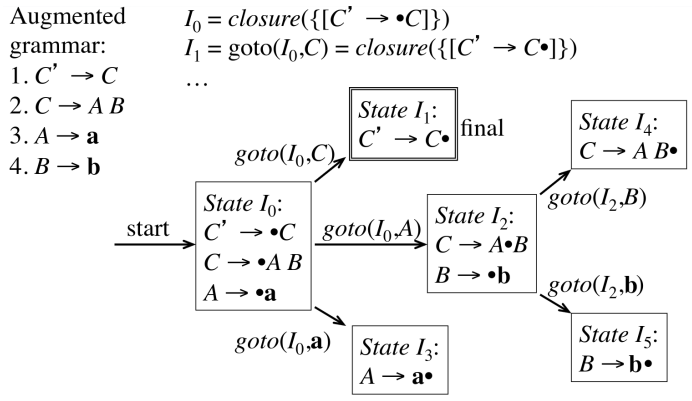


Figura 46: Esempio di DFA per la costruzione della tabella di parsing SLR

è il seguente:

	a	b	$	C	A	B
I_0	s3			1	2	
I_1			acc			
I_2	s5					4
I_3	r3	r3		1	2	
I_4		r2	r1	1	2	
I_5		r4	r2	1	2	

Tutte le riduzioni si fanno per i FOLLOW(state) e si effettua una reduce quando il pallino è alla fine della stringa, dove le riduzioni rappresentano:

1. $\mathrm{r1} : \mathrm{C}' \to \mathrm{C}$
2. $\mathrm{r2} : \mathrm{C} \to \mathrm{AB}$
3. $\mathrm{r3} : \mathrm{A} \to \mathrm{a}$
4. $\mathrm{r4} : \mathrm{B} \to \mathrm{b}$

Esempio 4.29. Un altro esempio di tabella di parsing SLR per la seguente grammatica:

$$(1)\mathrm{E} \to \mathrm{E} + \mathrm{T}$$
$$(2)\mathrm{E} \to \mathrm{T}$$
$$(3)\mathrm{T} \to \mathrm{T} * \mathrm{F}$$
$$(4)\mathrm{T} \to \mathrm{F}$$
$$(5)\mathrm{F} \to (\mathrm{E})$$
$$(6)\mathrm{F} \to \mathtt{id}$$

è il seguente:

STATE	ACTION						GOTO		
	id	+	*	(	)	$	*E*	*T*	*F*
0	s5			s4			1	2	3
1		s6				acc			
2		r2	s7		r2	r2			
3		r4	r4		r4	r4			
4	s5			s4			8	2	3
5		r6	r6		r6	r6			
6	s5			s4				9	3
7	s5			s4					10
8		s6			s11				
9		r1	s7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			

Tabella 6: Tabella di parsing

Eseguiamo il parsing della stringa `id * id + id` usando questa tabella:

	STACK	SYMBOLS	INPUT	ACTION
(1)	0		**id** $*$ **id** $+$ **id** \$	shift
(2)	0 5	**id**	$*$ **id** $+$ **id** \$	reduce by $F \to$ **id**
(3)	0 3	F	$*$ **id** $+$ **id** \$	reduce by $T \to F$
(4)	0 2	T	$*$ **id** $+$ **id** \$	shift
(5)	0 2 7	$T*$	**id** $+$ **id** \$	shift
(6)	0 2 7 5	$T*$ **id**	$+$ **id** \$	reduce by $F \to$ **id**
(7)	0 2 7 10	$T*F$	$+$ **id** \$	reduce by $T \to T*F$
(8)	0 2	T	$+$ **id** \$	reduce by $E \to T$
(9)	0 1	E	$+$ **id** \$	shift
(10)	0 1 6	$E+$	**id** \$	shift
(11)	0 1 6 5	$E+$ **id**	\$	reduce by $F \to$ **id**
(12)	0 1 6 3	$E+F$	\$	reduce by $T \to F$
(13)	0 1 6 9	$E+T$	\$	reduce by $E \to E+T$
(14)	0 1	E	\$	accept

Tabella 7: Parsing della stringa `id * id + id`

4.9.9 Parser LR(1)

La tabella di parsing SLR è deterministica e quindi richiede che non ci siano conflitti, di conseguenza tante grammatiche non sono SLR, incluse tutte le grammatiche ambigue. Siccome SLR è troppo semplice si introducono i parser LR(1) in cui ogni item è arricchito con un **token di lookahead**. Ad esempio:

$$[A \to \alpha \cdot \beta, a] \quad a \in \text{FOLLOW}(A)$$

In questo modo si riescono a risolvere i conflitti.

- Per item del tipo
$$[A \to \alpha\cdot, a]$$
Questo indica che la riduzione $A \to \alpha$ è applicabile soltanto se il token di lookahead a è presente nell'input.

- Per item del tipo:
$$[A \to \alpha \cdot \beta, a]$$
con $\beta \neq \varepsilon$ il simbolo di lookahead a è irrilevante

Ridichiariamo le funzioni `CLOSURE` e `GOTO` per i parser LR(1):

- `CLOSURE`:
  1. Inizialmente aggiungere ogni item in I a `CLOSURE(I)`
  2. Se $[A \to \alpha \cdot B\beta, a]$ è in `CLOSURE(I)` allora per ogni produzione $B \to \gamma$ nella grammatica e per ogni terminale $b \in \text{FIRST}(\beta a)$ aggiungere l'item $[B \to \cdot\gamma, b]$ a `CLOSURE(I)` se non è già presente
  3. Ripetere il passo 2 finchè non è più possibile aggiungere item a `CLOSURE(I)`

- `GOTO`:

1. Per ogni item $[A \to \alpha \cdot X\beta, a]$ in I aggiungere l'insieme di item $\texttt{CLOSURE}(\{[A \to \alpha X \cdot \beta, a]\})$ a $\texttt{GOTO}(I, X)$ se non è già presente
2. Ripetere il passo 1 finchè non è più possibile aggiungere item a $\texttt{GOTO}(I, X)$

Per costruire l'insieme degli item $LR(1)$ si utilizzano i seguenti passi:

1. Aumentare la grammatica con un nuovo simbolo iniziale S' e una produzione $S' \to S$
2. Inizialmente, impostare $C = \texttt{CLOSURE}(\{[S' \to \cdot S, \$]\})$ (questo è lo stato iniziale del DFA)
3. Per ogni insieme di item $I \in C$ e per ogni simbolo della grammatica X tale che $\texttt{GOTO}(I, X) \notin C$ e $\texttt{GOTO}(I, X) \neq \varnothing$, aggiungere l'insieme di item $\texttt{GOTO}(I, X)$ a C
4. Ripetere il passo 3 finchè non è più possibile aggiungere insiemi a C

Per costruire la tabella di parsing $LR(1)$ si utilizza il seguente algoritmo:

1. Costruire l'insieme $C = \{I_0, I_1, \ldots, I_n\}$ di insiemi di item $LR(1)$ per la grammatica G'
2. Le azioni di parsing per lo stato i sono determinate come segue:
   - Se $[A \to \alpha \cdot a\beta, b] \in I_i$ e $GOTO(I_i, a) = I_j$ allora impostare $ACTION[i, a] = shift\, j$
   - Se $[A \to \alpha\cdot, a] \in I_i$ e $A \neq S'$ allora impostare $ACTION[i, a] = reduce\, A \to \alpha$
   - Se $[S' \to S\cdot, \$] \in I_i$ allora impostare $ACTION[i, \$] = accept$
3. Se $GOTO(I_i, A) = I_j$ allora impostare $GOTO[i, A] = j$
4. Tutte le entry non definite dalle regole (2) e (3) sono impostate a `error`
5. Lo stato iniziale i è lo stato I_i che contiene l'item $[S' \to \cdot S, \$]$

Esempio 4.30. Consideriamo la grammatica:

$$S' \to S$$
$$S \to CC$$
$$C \to cC \mid d$$

1. Calcoliamo $\texttt{CLOSURE}(\{[S' \to \cdot S, \$]\})$:
   - Per prima cosa aggiungiamo $[S' \to \cdot CC, \$]$
   - Poi siccome abbiamo $FIRST(S) = FIRST(C) = \{c, d\}$ aggiungiamo:
     - $[C \to \cdot cC, c]$
     - $[C \to \cdot cC, d]$
     - $[C \to \cdot d, c]$
     - $[C \to \cdot d, d]$

- Nessuno dei nuovi item ha un non terminale immediatamente a destra del punto, quindi abbiamo completato il primo insieme di item LR(1)

Alla fine otteniamo:

$$\begin{aligned} \mathrm{I}_0 : \mathrm{S} &\to \cdot \mathrm{S}, \$ \\ \mathrm{S} &\to \cdot \mathrm{CC}, \$ \\ \mathrm{C} &\to \cdot \mathrm{cC}, \mathrm{c} \mid \mathrm{d} \\ \mathrm{C} &\to \cdot \mathrm{d}, \mathrm{c} \mid \mathrm{d} \end{aligned}$$

2. Calcoliamo $\texttt{GOTO}(\mathrm{I}_0, \mathrm{S})$: Bisogna calcolare $\texttt{CLOSURE}(\{[\mathrm{S}' \to \mathrm{S}\cdot, \$]\})$ che equivale all'elemento stesso siccome il punto è alla fine della stringa

3. Poi bisogna calcolare $\texttt{GOTO}(\mathrm{I}_0, \mathrm{C})$: Bisogna calcolare $\texttt{CLOSURE}(\{[\mathrm{S} \to \mathrm{C} \cdot \mathrm{C}, \$]\})$ che porta agli item di I_2:

$$\begin{aligned} \mathrm{I}_2 : \mathrm{S} &\to \mathrm{C} \cdot \mathrm{C}, \$ \\ \mathrm{C} &\to \cdot \mathrm{cC}, \$ \\ \mathrm{C} &\to \cdot \mathrm{d}, \$ \end{aligned}$$

4. Continuare a calcolare `GOTO` per tutti i simboli della grammatica e per tutti gli insiemi di item finchè non si ottiene la collezione canonica completa

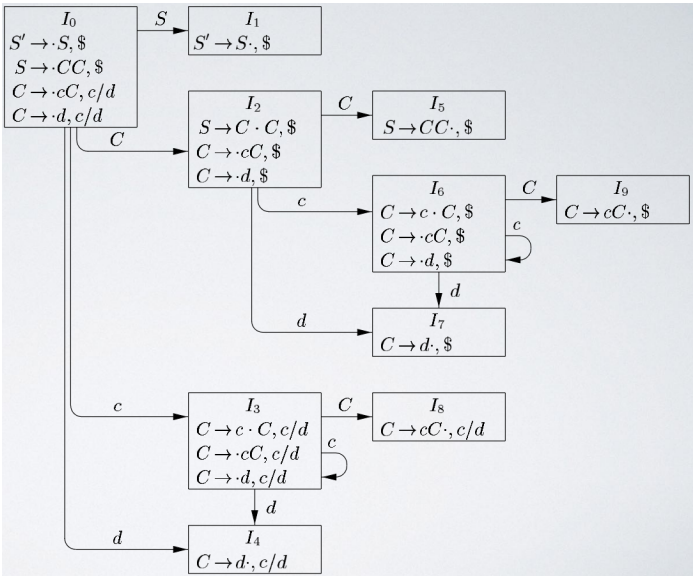


Figura 47: Esempio di collezione canonica per una grammatica LR(1)

Considerando le seguenti numerazioni delle produzioni:

$$
\begin{aligned}
&S' \to S \\
(r1)\ &S \to CC \\
(r2)\ &C \to cC \\
(r3)\ &C \to d
\end{aligned}
$$

la tabella di parsing LR(1) è la seguente:

STATE	ACTION			GOTO	
	c	d	\$	S	C
0	s3	s4		1	2
1			acc		
2	s6	s7			5
3	s3	s4			8
4	r3	r3			
5			r1		
6	s6	s7			9
7			r3		
8	r2	r2			
9			r2		

4.9.10 Parser LALR

Le tabelle di parsing LR(1) hanno tanti stati, quindi per ridurli si introduce il parser LALR (Look-Ahead LR) che unisce due o più stati LR(1) che hanno lo stesso **core**, cioè il primo componente dell'item. Nell'esempio 4.30 gli stati I_4 e I_7 hanno lo stesso core. Siccome GOTO(I, X) dipende solo dal core di I allora il GOTO di stati uniti è l'unione dei GOTO dei singoli stati. Questo parser introduce degli svantaggi:

- Meno potente del LR(1)
- L'unione degli stati può introdurre conflitti:
  - Non vengono introdotti conflitti di shift perchè gli shift non usano il token di lookahead
  - **Potrebbero** essere introdotti conflitti di reduce, ma questo succede raramente per grammatiche di linguaggi di programmazione

Per costruire la tabella di parsing LALR si utilizza il seguente algoritmo:

1. Costruire l'insieme $C = \{I_0, I_1, \ldots, I_n\}$ di insiemi di item LR(1) per la grammatica G′
2. Per ogni core presente tra gli insiemi di item LR(1), trovare tutti gli insiemi che hanno quel core, e sostituire questi insiemi con la loro unione

3. Sia $C = \{J_0, J_1, \ldots, J_m\}$ l'insieme risultante di insiemi di item uniti. Le azioni di parsing per lo stato i sono costruite da J_i seguendo la procedura per la tabella di parsing LR(1)

4. Sia $J = I_0 \cup I_1 \cup \ldots \cup I_k$ allora

$$GOTO(I_0, X) = GOTO(I_1, X) = \ldots = GOTO(I_k, X)$$

siccome hanno lo stesso core. Sia K l'insieme di tutti gli item che hanno lo stesso core di $GOTO(I_1, X)$, allora $GOTO(J, X) = K$

Esempio 4.31. Consideriamo la grammatica dell'esempio 4.30 e la collezione canonica LR(1) con tutti gli stati che hanno lo stesso core evidenziati:

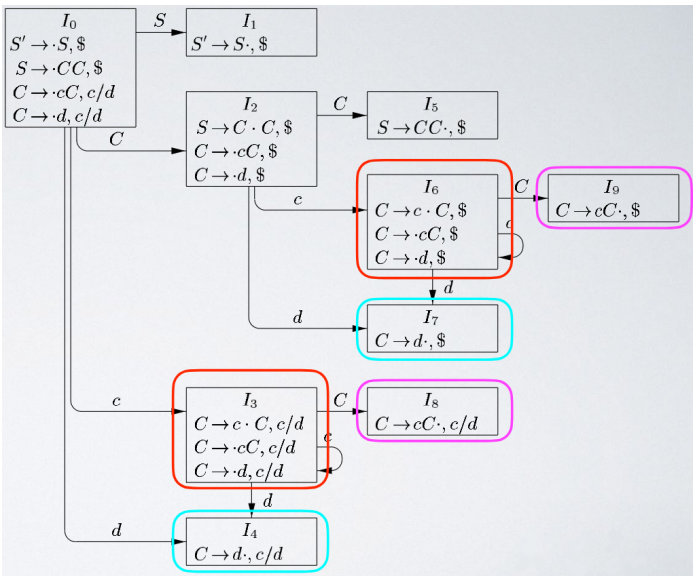


Figura 48: Collezione canonica LR(1) con stati che hanno lo stesso core evidenziati

L'unione degli stati con lo stesso core produce i seguenti nuovi stati:

$$I_{36} : C \to c \cdot C, c \mid d \mid \$$$$C \to \cdot cC, c \mid d \mid \$$$$C \to \cdot d, c \mid d \mid \$$$

$$I_{47} : \; C \to d\cdot, c \mid d \mid \$$$

$$I_{89} : \; C \to cC\cdot, c \mid d \mid \$$$

La nuova tabella di parsing LALR è la seguente:

STATE	ACTION			GOTO	
	c	d	\$	S	C
0	s36	s47		1	2
1			acc		
2	s36	s47			5
36	s36	s47			89
47	r3	r3	r3		
5			r1		
89	r2	r2	r2		

Tabella 8: Tabella di parsing LALR

4.9.11 Regole di precedenza e associatività

Le grammatiche ambigue forniscono una specifica più naturale, ma non sono LR, quindi sono necessarie delle regole che eliminino le ambiguità e permettano di costruire un parse tree univoco per la grammatica.

Esempio 4.32. La seguente grammatica per le espressioni aritmetiche è ambigua e non specifica regole di precedenza e associatività:

$$\mathsf{E} \to \mathsf{E} + \mathsf{E} \mid \mathsf{E} * \mathsf{E} \mid (\mathsf{E}) \mid \texttt{id}$$

La seguente grammatica equivalente specifica la precedenza di $*$ su $+$ e l'associatività a sinistra di entrambi gli operatori:

$$\mathsf{E} \to \mathsf{E} + \mathsf{T} \mid \mathsf{T}$$
$$\mathsf{T} \to \mathsf{T} * \mathsf{F} \mid \mathsf{F}$$
$$\mathsf{F} \to (\mathsf{E}) \mid \texttt{id}$$

Il parser per la grammatica non ambigua è più complesso e quindi meno efficiente perchè la maggior parte del tempo è speso a ridurre le produzioni unitarie $\mathsf{E} \to \mathsf{T}$ e $\mathsf{T} \to \mathsf{F}$ che hanno la sola funzione di specificare la precedenza.